Block glossary

Every block in the mBlock palette for this classroom's robot — 245 of them — grouped the way the toolbox groups them. Each one says what it does, when you would reach for it, and shows a working example.

Before programming: Live Mode versus Upload Mode

The mode switch in mBlock decides **where the program runs**. It is not merely a different way to send the same program.

	Live Mode	Upload Mode
Where code runs	On the computer; commands are sent to the connected robot.	On the CyberPi inside the robot.
Connection	The robot must remain connected and mBlock must stay open.	After uploading, the cable and computer can be disconnected.
How it starts	The IDE's green flag, keyboard events or clicking a script can start it.	**when CyberPi starts up** and the robot's buttons or sensors start scripts.
Where output appears	The IDE stage can show Panda, variables and reporter monitors.	Use the physical CyberPi screen, LEDs and speaker.
Best for	Quick desk tests and watching values while connected.	An autonomous robot that drives away from the computer.

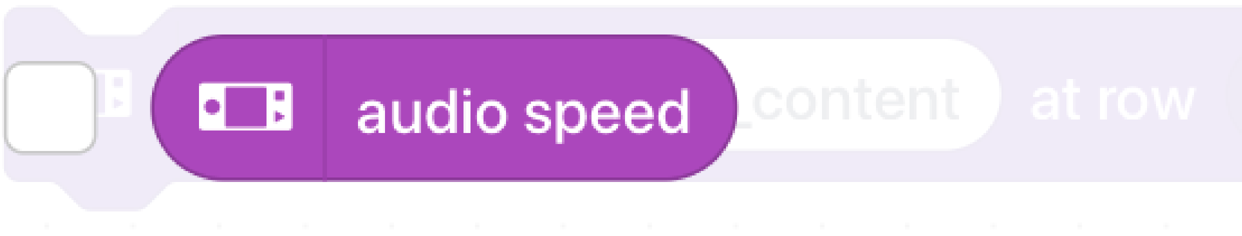
This course normally uses Upload Mode. Uploading translates the blocks, replaces the previous program stored on the CyberPi, and restarts it. The robot can then run by itself. Entries marked **Live Mode Only** rely on the computer or its IDE stage and are exceptions.

```
Live = the computer directs the robot now
Upload = give the robot the whole program, then let it work alone
```

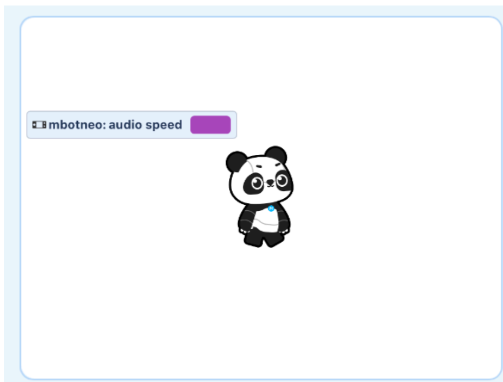
Watching a value in the mBlock IDE

Some rounded value blocks have a small checkbox beside them in the Blocks drawer. Checking it creates a **monitor**: a small readout in the top-left of the mBlock stage, the white area containing the Panda. The readout shows the value currently reported by that block and updates while mBlock can read it.

This is a testing and debugging display in the **IDE on the computer**. It is not shown on the CyberPi's physical screen and it is not added to the uploaded robot program. Hardware readings normally require the device to be connected while testing in Live mode. Clear the checkbox to remove the monitor from the stage.



1. Check the box beside a watchable value block in its drawer.



2. Its named monitor appears at the top-left of the IDE stage and displays the current value.

Events

Hat blocks. Each one starts a script when something happens, and a script can only have one. Which hat you choose decides when — and whether — your program runs at all.

when CyberPi starts up

WHAT IT DOES

A hat block. Everything under it runs the moment the CyberPi powers on or is reset — and, because an upload restarts the robot, the moment a program finishes uploading.

USE IT WHEN

For setup that should happen once and for anything the robot should do unattended. In this course it usually carries a single line telling the driver what to press, because a program that drives the moment it boots is a program that drives off the table during an upload.

EXAMPLE

when CyberPi starts up
show label 1 ▾ press A to start at center of screen ▾ by small ▾ pixel

On power-up, print the instruction and wait. Nothing moves until a person decides it should.

> Only one script may sit under any one hat. Two "when CyberPi starts up" hats in the same project is not two programs — it is a crash.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G7 step 8, G7 step 9 · See also **when button A pressed**

when joystick pulled↑ ▾

WHAT IT DOES

A hat block for the CyberPi's own five-way joystick — the little stick on the board itself. It fires when the stick is pushed up, down, left or right, or pressed in the middle.

Possible dropdown values

Menu shown as "pulled↑"

`pulled↑` `pulled↓` `pulled←`
`pulled→` `middle pressed`

USE IT WHEN

For choosing between things without extra hardware: a menu, a mode switch, a nudge in one direction. Handy when every group has a robot but not every group has a gamepad.

EXAMPLE

when joystick pulled↑ ▾
moves forward ▾ at 40 RPM for 1 secs

Push the onboard stick up and the robot takes one step forward.

> This is the stick built into the CyberPi, not the Bluetooth gamepad. The gamepad has its own blocks in the Bluetooth controller drawer.

Used in G7 step 4, G7 step 5, G7 step 6, G7 step 14, G7 step 15, G7 step 16 · See also **joystick RX**

when button A ▾ pressed

WHAT IT DOES

A hat block that runs its script when button A or button B on the CyberPi is pressed.

Possible dropdown values

Menu shown as “A”

`A` `B`

USE IT WHEN

This is the course's standard start button. Uploading always runs the program immediately, so putting the moving part under button A is what keeps the robot still until somebody is ready.

EXAMPLE

```
when button A ▾ pressed
moves forward ▾ 20 cm ▾ until done
```

The robot waits, however long it takes, until a person presses A.

> A and B are the two round buttons beside the screen, not the joystick.

● Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G7 step 9, G7 step 10 · See also **when CyberPi starts up**, **when joystick pulled↑**

when CyberPi tilted left ▾

WHAT IT DOES

A hat block that fires when the CyberPi is being held at a particular angle — tilted left, right, forward, back, or lying screen up or screen down.

Possible dropdown values

Menu shown as “tilted left”

`tilted left` `tilted right` `tilted forward` `tilted backward` `screen faces up` `screen faces down`

USE IT WHEN

When the robot itself is the controller, or to notice that something has gone wrong: a robot that reports "screen faces down" has been knocked over.

EXAMPLE

```
when CyberPi tilted left ▾
play hi ▾
```

Tip the robot to the left and it answers with a sound.

See also **when wave left detected**

WHAT IT DOES

A hat block that fires when the CyberPi itself is moved quickly in the chosen direction, or twisted clockwise or anticlockwise. Here, "wave left" means flicking the CyberPi to the left; it does not mean waving your hand in front of the robot.

Possible dropdown values

Menu shown as “wave left”

`wave left` `wave right` `wave up` `wave down` `rotate clockwise` `rotate countercloclwise` `freefall` `shaken`

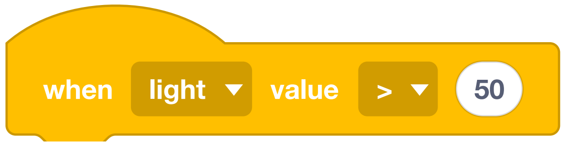
USE IT WHEN

When you want the CyberPi's internal motion sensor to react to a physical gesture. It detects a sharp movement, so it suits "flick left to switch off" better than it suits precise control.

EXAMPLE

Quickly move the CyberPi to the left to switch the lights off.

See also **when CyberPi tilted left**



WHAT IT DOES

A hat block that watches one built-in sensor — light, sound, shaking or the timer — and fires the instant it crosses a number you set.

Possible dropdown values

Menu shown as “light”

`light` `sound` `shaking strength` `timer`

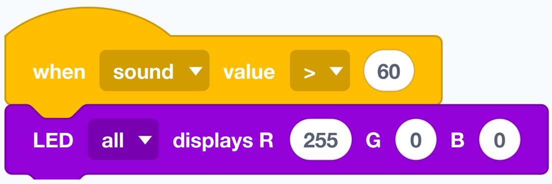
Menu shown as “>”

`>` `<`

USE IT WHEN

When you want a reaction without writing the watching yourself. It replaces a forever loop with an if inside it, and it keeps running in the background while your main script does something else.

EXAMPLE



A loud clap turns the ring red, whatever else the robot is doing.

> It fires on the crossing, not while the value stays high — a continuous loud noise triggers it once, not over and over.

Used in G7 step 11 · See also **loudness**, **ambient light intensity**

LIVE MODE ONLY

when green flag clicked

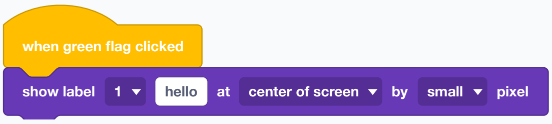
WHAT IT DOES

The Scratch green-flag hat. It runs when you click the green flag button in the mBlock IDE (the programming screen), above the stage on the computer.

USE IT WHEN

Only in Live mode, where the program runs on the computer and the robot is a puppet on the end of a cable. An uploaded program has no flag to click, so this hat never fires on the robot.

EXAMPLE



In Live mode, clicking the flag writes on the robot's screen.

> If your program does nothing after an upload, check that its hat is not this one. Use "when CyberPi starts up" or "when button A pressed" instead.

See also **when CyberPi starts up**

LIVE MODE ONLY

when space ▾ key pressed

WHAT IT DOES

A hat block that fires when a key on the computer keyboard is pressed.

Possible dropdown values

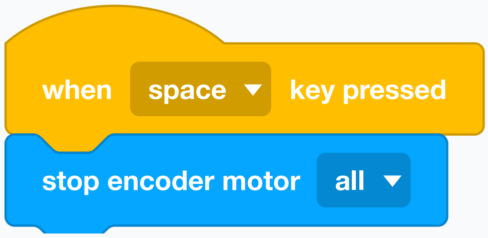
Menu shown as “space”

`space` `up arrow` `down arrow` `right arrow` `left arrow` `any`

USE IT WHEN

Live mode only, for the same reason as the green flag — an uploaded robot is not attached to your keyboard. Useful for quick testing at the desk.

EXAMPLE



A space bar panic button while you are testing in Live mode.

See also **when green flag clicked**

when I receive

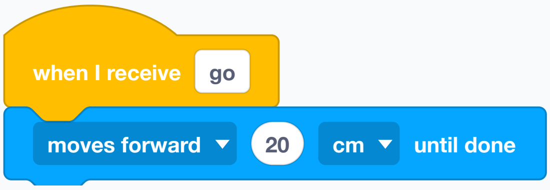
WHAT IT DOES

A hat block that runs when it receives a broadcast: a named signal sent from another part of the same program. A broadcast does not contain code itself; it simply tells every matching "when I receive" script to start.

USE IT WHEN

To split a program into named parts that call each other, and to let one event start several scripts at once. Every script with the same "when I receive" runs together. Broadcasts are introduced later in the course, so it is fine if you have not used this idea yet.

EXAMPLE



Anything that broadcasts "go" sets this script running.

See also **broadcast**, **broadcast and wait**

broadcast

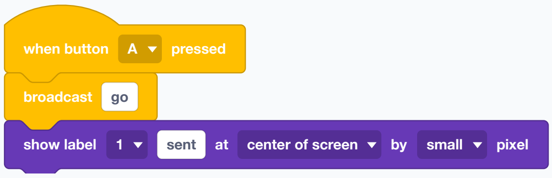
WHAT IT DOES

Sends a named message, and carries straight on without waiting for anything that answers it.

USE IT WHEN

When one thing should trigger another but the two do not have to take turns — starting the lights while the driving continues.

EXAMPLE



The message goes out and this script keeps moving immediately.

See also **when I receive**, **broadcast and wait**

broadcast and wait

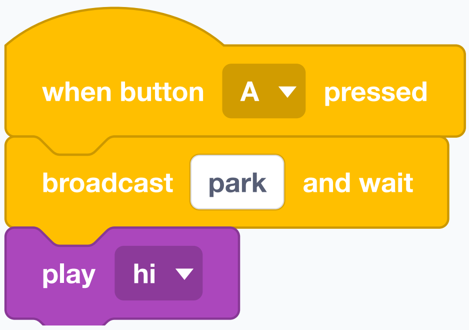
WHAT IT DOES

Sends a named message and then holds this script until every script listening for that message has finished.

USE IT WHEN

When order matters — do that, and only when it is completely done, do this.

EXAMPLE



Park first; only when parking is finished does the robot chirp.

See also **broadcast**

Control

The shape of a program: what repeats, what only happens sometimes, and what waits. These blocks hold other blocks rather than doing anything themselves.

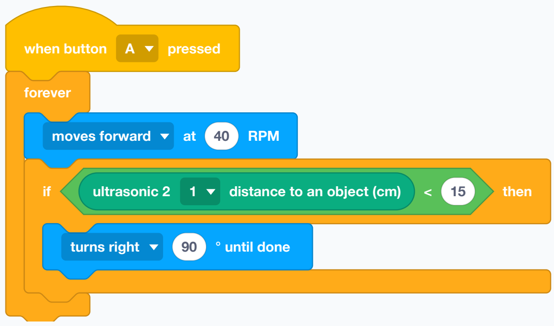
WHAT IT DOES

A loop with no end. Everything inside it runs, then runs again, for as long as the program lives.

USE IT WHEN

For anything the robot must keep doing: watching a sensor, following a line, answering a controller. Almost every robot program has one at its heart.

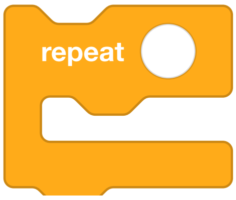
EXAMPLE



Drive and keep looking. Every time a wall comes close, turn away from it.

Nothing after a forever loop ever runs, because the loop never ends. If you have blocks below one, they are dead.

Used in G7 step 6, G7 step 8, G7 step 9, G7 step 10, G7 step 11, G7 step 13 · See also **repeat**, **repeat until**



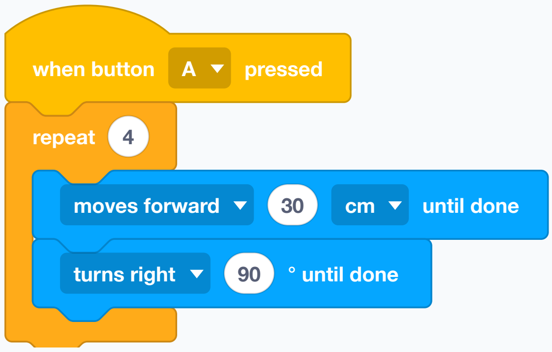
WHAT IT DOES

Runs everything inside it a fixed number of times, then carries on to whatever comes next.

USE IT WHEN

When you know the count in advance — four sides of a square, three beeps, ten steps.

EXAMPLE



A square, written once and run four times.

Used in G7 step 5, G7 step 6, G7 step 18, G7 step 19, G7 step 20, G7 step 21 · See also **count with from to by step repeat**, **forever**

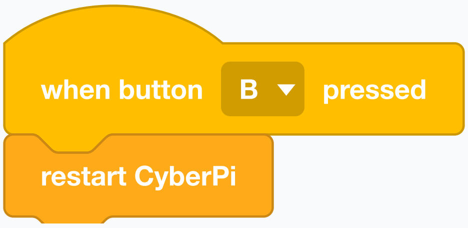
WHAT IT DOES

Reboots the CyberPi, as though it had been switched off and on again.

USE IT WHEN

Rarely. To return to a clean state after something has gone wrong, or to restart a program from its very beginning.

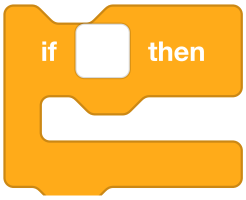
EXAMPLE



A software reset button.

> Everything in memory is lost — every variable goes back to its starting value.

See also **stop all**



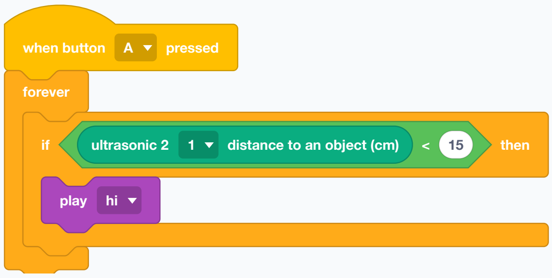
WHAT IT DOES

Runs the blocks inside it once, but only if the condition is true right now. If it is false, they are skipped.

USE IT WHEN

For a decision with nothing to do on the other side. Inside a forever loop it becomes "check this, every time round".

EXAMPLE



Beep whenever something is close. Say nothing when it is not.

> It tests once, at the moment the script reaches it. To keep testing, put it inside a loop.

Used in G7 step 14, G7 step 15, G7 step 16, G7 step 18, G7 step 19, G7 step 20 · See also **if then**, **repeat until**

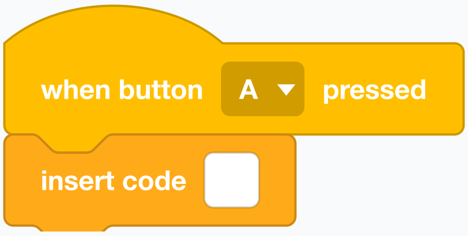
WHAT IT DOES

Lets you type a line of Python straight into a block program, which runs where the block sits.

USE IT WHEN

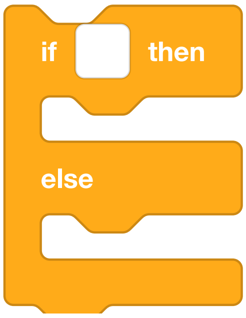
When you need something the palette does not offer. It is the bridge between the block course and the Python course, and it is how a Grade 8 class peeks at what Grade 9 does.

EXAMPLE



One block, holding whatever line of Python you type into it.

Upload mode only, and a mistyped line fails on the robot rather than in the editor.



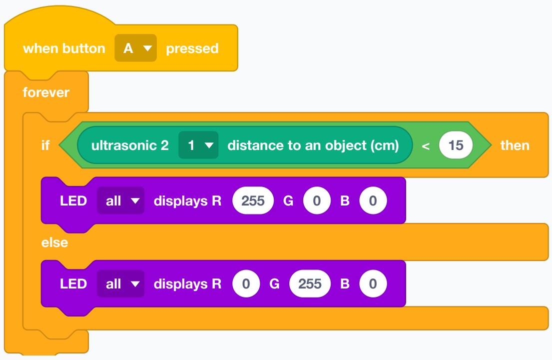
WHAT IT DOES

Two paths, one decision. The top half runs when the condition is true, the bottom half when it is false — always one, never both.

USE IT WHEN

When there is genuinely something to do either way: go left or go right, green light or red light.

EXAMPLE



Red when the way is blocked, green when it is clear — and never neither.

● Used in G7 step 9, G7 step 10, G7 step 11, G7 step 13, G7 step 14, G7 step 15 · See also **if then**

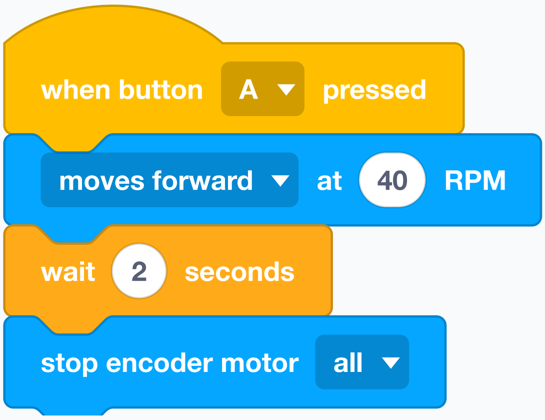
WHAT IT DOES

Holds the script still for a number of seconds, then carries on.

USE IT WHEN

To let something finish that has no "until done" of its own, to space out a display so a person can read it, or to slow a loop that is spinning far faster than anything can react to.

EXAMPLE



Drive for exactly two seconds, then stop. The wait is what makes it two.

Used in G8 step 3, G8 step 4, G8 step 5, G8 step 7, G8 step 8, G8 step 9 · See also **wait until**, **moves forward at RPM for secs**

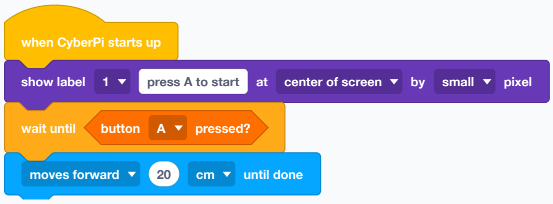
WHAT IT DOES

Holds the script still until a condition becomes true, however long that takes, then carries on.

USE IT WHEN

When you want to wait for a thing rather than for a time — until a button is pressed, until a wall is close, until a line appears.

EXAMPLE



The pattern that stops an uploaded program driving off the table: it waits for a person before it moves.

> If the condition is never true, the script waits for ever. Give yourself a way out.

Used in G7 step 6, G7 step 13, G7 step 14, G7 step 15, G7 step 16, G7 step 19 · See also **repeat until**, **wait seconds**

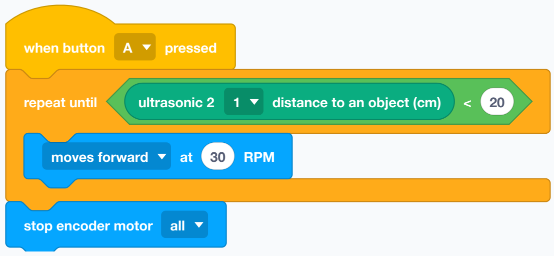
WHAT IT DOES

Keeps looping until a condition becomes true. It checks BEFORE each pass, so if the condition is already true the inside never runs at all.

USE IT WHEN

When the count is unknown but the finish line is clear: keep creeping forward until the wall is close, keep turning until the line is found.

EXAMPLE



Creep forward for as long as the way is clear, and stop when it is not.

● Used in G7 step 10, G7 step 11, G7 step 24 · See also **wait until**, **forever**

WHAT IT DOES

Stops running scripts. "all" stops the entire program, "this script" stops only the one the block sits in.

Possible dropdown values

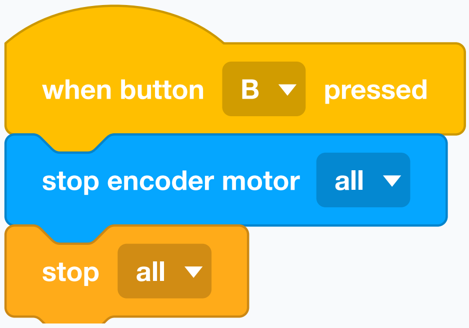
Menu shown as “all”

`all` `this script` `other scripts in sprite`

USE IT WHEN

To end a program deliberately — a finish line reached, a task complete.

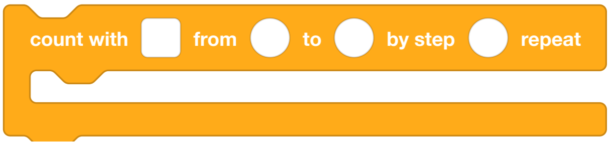
EXAMPLE



B is the emergency stop: cut the motors, then stop the program.

> Stopping the program does not stop the wheels. Motors keep their last order, so stop them first.

● Used in G7 step 6, G7 step 24, G8 step 9, G8 step 10 · See also **stop encoder motor (1) EM1**, **break**



WHAT IT DOES

A counting loop. It makes a variable, counts it from a start to an end in steps you choose, and runs the inside once per count — so the blocks inside can use the number.

USE IT WHEN

When the repeat number is also useful inside the loop: walking through a list item by item, drawing a shape whose sides grow, printing a numbered sequence.

EXAMPLE

Four legs, and the loop knows which leg it is on.

> The end number is not included. To walk a whole list, count to one past its length.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **repeat**, **item of my list**

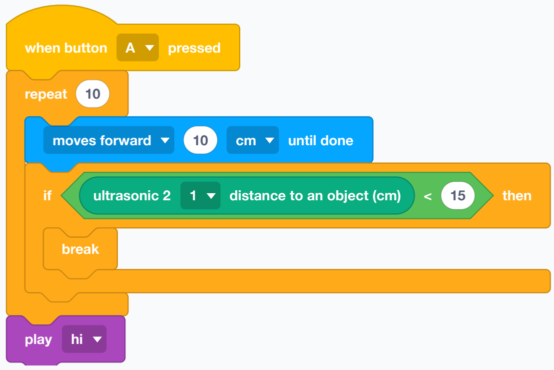
WHAT IT DOES

Leaves the loop it is inside immediately and carries on after it.

USE IT WHEN

When something inside a loop decides there is no point going round again — the thing was found, the limit was hit.

EXAMPLE



Up to ten steps, but stop early if a wall turns up — and chirp either way.

● Used in G7 step 16, G8 step 18, G8 step 19, G8 step 20 · See also **continue**, **stop all**

continue

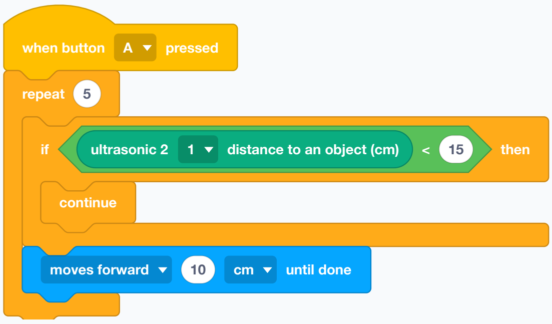
WHAT IT DOES

Skips the rest of this time round the loop and starts the next one.

USE IT WHEN

When one case has nothing to do but the loop should carry on — stepping past an item you do not care about.

EXAMPLE



Take a step, unless the way is blocked, in which case just look again.

See also **break**

Operators

Arithmetic and questions. None of these do anything on their own — they are reporters, and they go INTO the slots of other blocks.

WHAT IT DOES

Converts a value from one kind to another — text into a whole number, a number into text.

Possible dropdown values

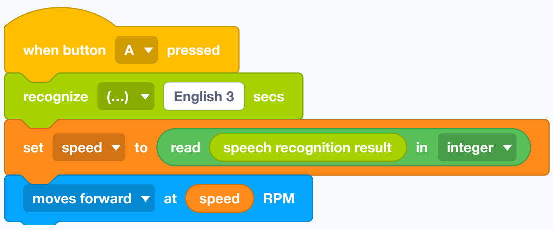
Menu shown as “integer”

`integer` `float` `string`

USE IT WHEN

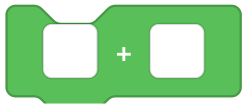
Only when something arrives as text but has to be used as a number. If you already have a numeric value, put it directly into the other block; no conversion is needed. Speech recognition, for example, returns text even when that text looks like a number.

EXAMPLE



Say "50". Speech recognition returns the text "50"; read converts it to the number 50 before the motor uses it as a speed.

See also **join**



WHAT IT DOES

Adds two numbers and reports the answer. It is a reporter, so it does not do anything by itself — you drop it into another block's slot.

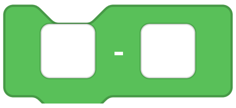
USE IT WHEN

Wherever a number has to be worked out rather than typed. Steering is the common case: a base speed plus a correction.

EXAMPLE

The slot is filled with a sum instead of a fixed number.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **-**, *****



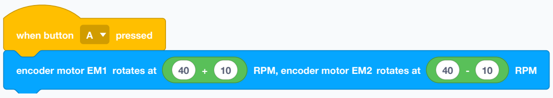
WHAT IT DOES

Takes one number away from another and reports the answer.

USE IT WHEN

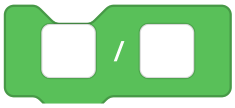
For differences — how far off a target you are, how much is left. A steering correction is usually one wheel plus the error and the other wheel minus it.

EXAMPLE



Same base speed, opposite corrections: the robot curves.

Used in G7 step 20, G7 step 21, G7 step 24, G8 step 9, G8 step 10 · See also +



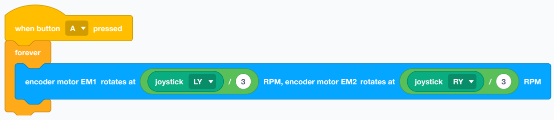
WHAT IT DOES

Divides the first number by the second and reports the answer.

USE IT WHEN

To shrink something that is too big. A joystick reading runs to about 100, which is a wild speed for a wheel, so it is nearly always divided down.

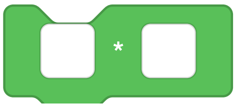
EXAMPLE



Full stick becomes a sensible speed rather than a bolt across the room.

> Dividing by zero has no answer. If the divisor comes from a sensor, make sure it can never be zero.

Used in G7 step 5, G7 step 6, G8 step 12, G8 step 13, G8 step 14, G8 step 15 · See also *



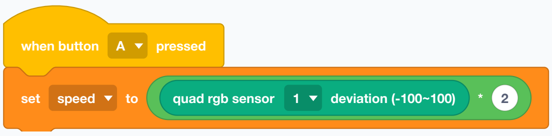
WHAT IT DOES

Multiplies two numbers and reports the answer.

USE IT WHEN

For scaling. A sensor reading rarely arrives in the units you want; multiplying turns it into a speed, an angle, or a brightness.

EXAMPLE



Take the line sensor's error and make the response twice as strong.

Used in G8 step 9, G8 step 10 · See also /

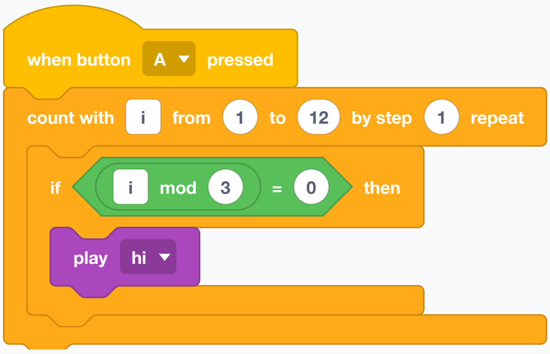
WHAT IT DOES

Reports the remainder after dividing — 7 mod 3 is 1, and 8 mod 4 is 0.

USE IT WHEN

For "every so often" and for wrapping round. A counter mod 4 cycles 0, 1, 2, 3, 0, 1 … which is exactly what you want for stepping through four colours or four sides.

EXAMPLE



A beep on every third pass, and silence on the other two.

See also **round**

WHAT IT DOES

Rounds a number to the nearest whole number.

USE IT WHEN

When a calculation gives a long decimal and only a whole number makes sense — a count, an index into a list, a number to show on the screen.

EXAMPLE

3.333… shown as 3.

See also **abs of**

WHAT IT DOES

Picks a whole number at random between the two you give, including both ends, and reports it. A new number every time it runs.

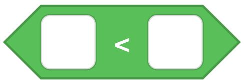
USE IT WHEN

To make behaviour unpredictable: a random turn when a wall appears, a random colour, a random wait. It is what turns a robot that repeats into a robot that explores.

EXAMPLE

A wander: every corner is a different corner.

Used in G7 step 6, G8 step 8, G8 step 9, G8 step 10 · See also **abs of**



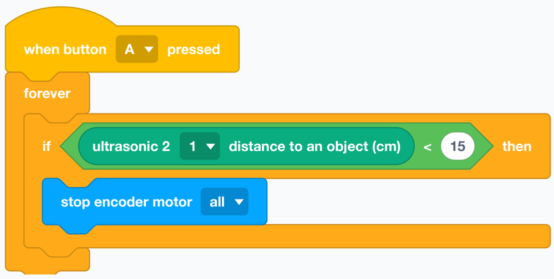
WHAT IT DOES

Reports true when the left number is smaller than the right, and false otherwise. It has a pointed shape because it belongs in a question slot.

USE IT WHEN

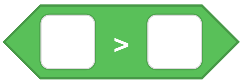
The everyday sensor test. "Closer than 15 cm" is a distance reading less than 15.

EXAMPLE



A wall is anything nearer than fifteen centimetres.

Used in G7 step 11 · See also **>, =**



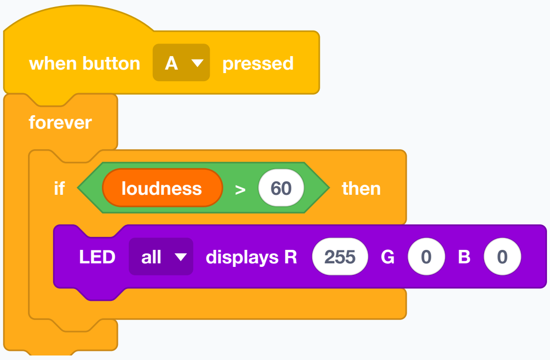
WHAT IT DOES

Reports true when the left number is bigger than the right.

USE IT WHEN

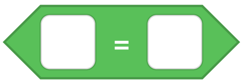
The other half of a threshold. Loud is a sound reading above a number; bright is a light reading above a number.

EXAMPLE



Flash red on any noise louder than 60.

See also **<**



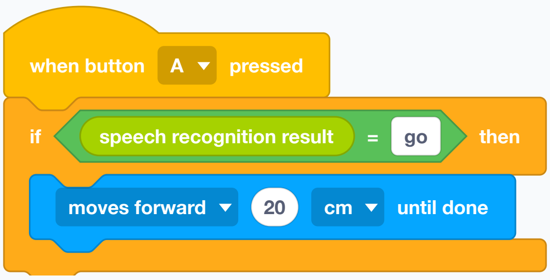
WHAT IT DOES

Reports true when the two things are exactly the same. It works on words as well as numbers.

USE IT WHEN

For matching an exact value — a command word that was heard, a mode number, a counter reaching its target.

EXAMPLE



Move only if the word heard was exactly "go".

> Exactly means exactly. A sensor reading that drifts by a hair will never equal a fixed number — use less-than or greater-than for those.

See also **contains ?**, **<**

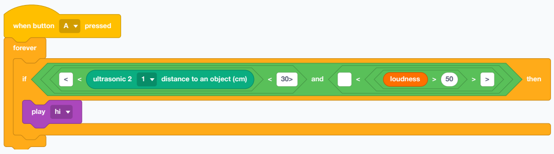
WHAT IT DOES

Reports true only when BOTH questions inside it are true.

USE IT WHEN

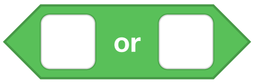
When two things have to hold at once — close enough AND on the line, button held AND moving.

EXAMPLE



Only answers when something is both near and noisy.

See also **or**, **not**



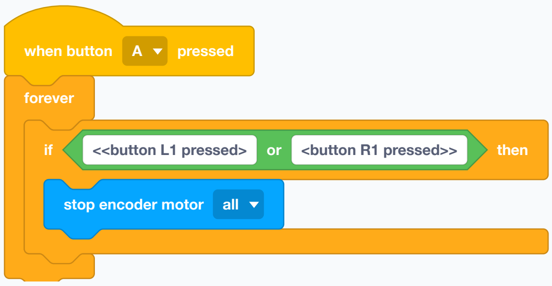
WHAT IT DOES

Reports true when EITHER question inside it is true — or both.

USE IT WHEN

When any one of several things should trigger the same response. Two bump sensors, either side.

EXAMPLE



Either shoulder button stops the robot.

See also **and**

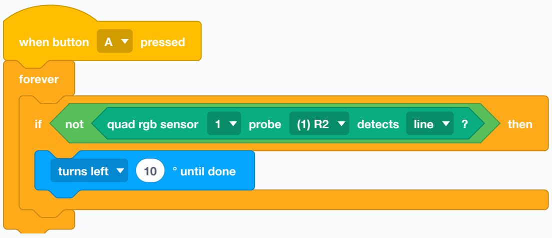
WHAT IT DOES

Flips a question over: true becomes false, false becomes true.

USE IT WHEN

When the thing you can ask about is the opposite of the thing you care about. "Not on the line" is easier to write than a list of every way to be off it.

EXAMPLE



Hunt for the line whenever the probe cannot see it.

See also **and**

WHAT IT DOES

Reports a single character out of a piece of text, counting from 1.

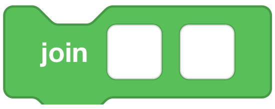
USE IT WHEN

Rarely in this course. It matters when a command arrives as text and only the first letter decides what to do.

EXAMPLE

The first letter of "forward" is f.

See also **length of**



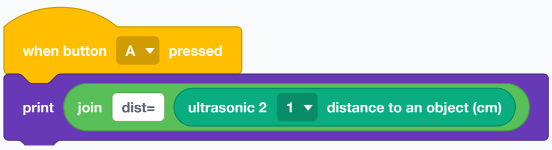
WHAT IT DOES

Sticks two pieces of text together and reports the result.

USE IT WHEN

For building a message out of a label and a value, so a screen reads "dist=23" instead of a bare 23.

EXAMPLE



A reading with its name attached, which is a great deal easier to read across a classroom.

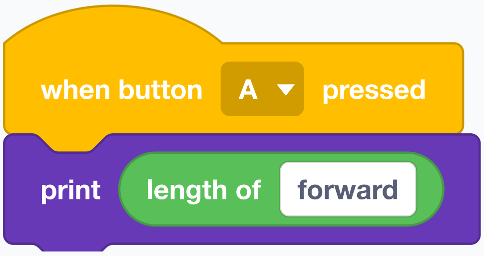
WHAT IT DOES

Reports how many characters are in a piece of text.

USE IT WHEN

When you need the size of a word rather than its content.

EXAMPLE



"forward" is seven characters long.

> This is the TEXT one. There is a second block worded almost identically in this same drawer, "length of my list", which counts list items. Putting the text block on a list name measures the NAME — a list called route reports 5, every time, whatever is in it.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **length of my list**

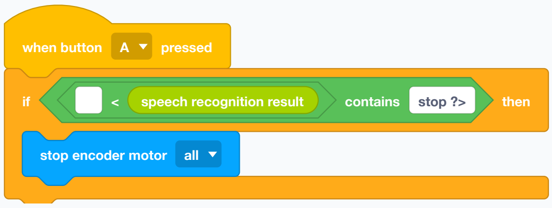
WHAT IT DOES

Reports true when the first piece of text has the second somewhere inside it.

USE IT WHEN

For loose matching, where equals is too strict — checking whether a heard sentence contains the word "stop" without demanding it be the only word.

EXAMPLE



"please stop now" contains "stop", so this triggers where equals would not.

● Used in G7 step 16, G8 step 18, G8 step 19, G8 step 20 · See also **=**, **my list contains ?**

WHAT IT DOES

A whole family of maths functions in one block — absolute value, floor, ceiling, square root, and the trigonometry ones — chosen from its dropdown.

Possible dropdown values

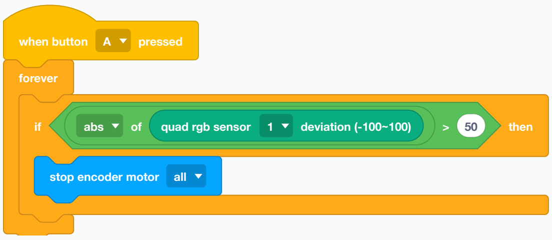
Menu shown as “abs”

`abs` `floor` `ceiling` `sqrt` `sin` `cos` `tan` `asin` `acos` `atan` `ln` `log` `e ^` `10 ^`

USE IT WHEN

Mostly "abs", which throws away the minus sign. A line sensor reporting -40 and one reporting 40 are equally far off; abs lets you ask "how far off" without caring which side.

EXAMPLE



Stop when the robot is badly off the line, whichever side it has drifted to.

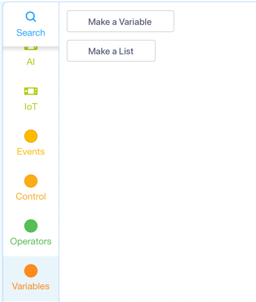
See also **round**

Variables

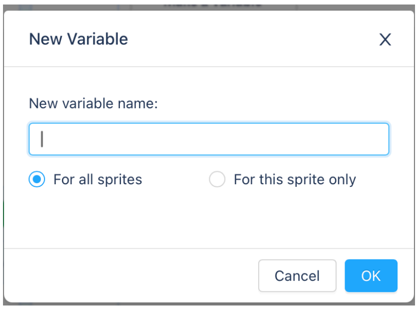
Named boxes. A variable holds one thing; a list holds a numbered row of things. Both are made in the drawer before they can be used.

Before using these blocks: create a variable or list

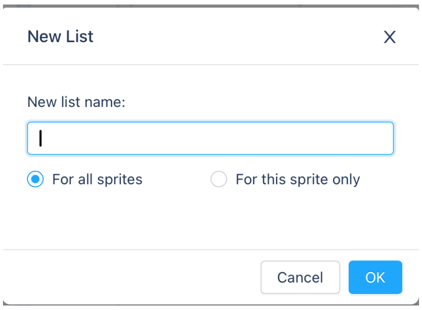
A variable stores one value. A list stores several values in numbered order.



1. Open the **Variables** drawer. Click **Make a Variable** or **Make a List**, depending on what you need to store.



2. Enter a clear variable name. Choose **For all sprites** to share it, or **For this sprite only** to keep it inside the current sprite. Then click **OK**.



3. Lists are created the same way: enter a name, choose its scope, and click **OK**. mBlock then adds blocks carrying that name to the Variables drawer.

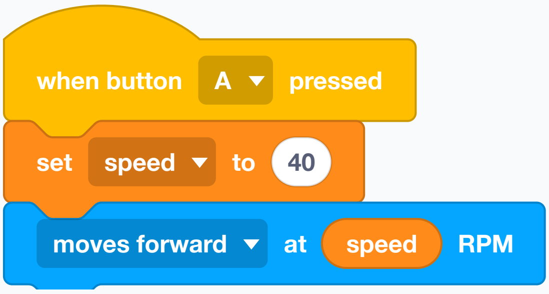
WHAT IT DOES

Puts a value into a variable, throwing away whatever was there before. A variable is a named box holding one thing. Use "set" when you know the exact value you want: setting speed to 40 makes it 40, whatever it was before.

USE IT WHEN

To give a variable its starting value, reset it, or replace it with a new measurement — a chosen speed, a threshold, a count, or the latest sensor reading.

EXAMPLE



Name the speed once, then use the name. Changing it later means changing one number in one place.

> Variables start empty each time an uploaded program runs. Set them before you read them.

Used in G7 step 5, G7 step 6, G7 step 13, G7 step 14, G7 step 15, G7 step 16 · See also **change my variable by**

WHAT IT DOES

Adds a number to what is already in a variable instead of replacing it. If speed is 40, "change speed by 5" makes it 45, while "set speed to 5" makes it exactly 5. A negative change subtracts: "change speed by -5" turns 40 into 35.

USE IT WHEN

When the new value depends on the old one: counting laps, counting how many times a sensor fired, or nudging a speed up or down while a button is held. Use "set" for a starting value, a reset, or an exact value; use "change by" for a counter or a gradual adjustment.

EXAMPLE

A lap counter: start at nothing, add one each time round, show the total.

Used in G7 step 13, G7 step 14, G7 step 15, G7 step 16, G8 step 8, G8 step 9 · See also **set my variable to**

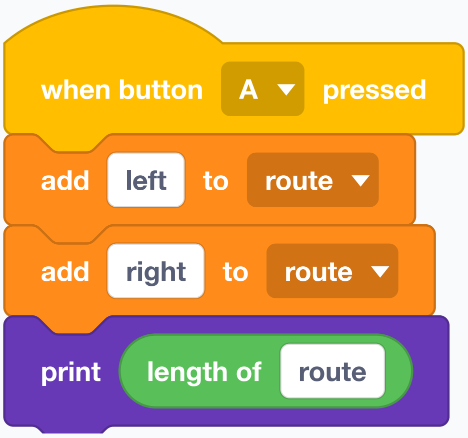
WHAT IT DOES

Puts a new item on the end of a list. A list is a numbered row of boxes rather than a single one.

USE IT WHEN

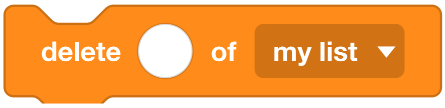
Whenever you are collecting rather than replacing — a route being recorded, a series of readings, a set of commands heard.

EXAMPLE



Two moves recorded, and the list now reports its own length.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **item of my list**, **length of my list**



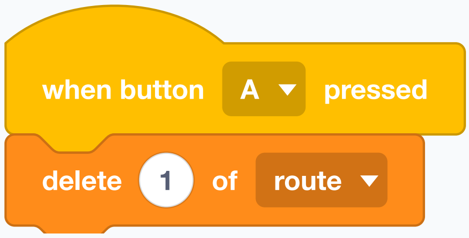
WHAT IT DOES

Removes one item from a list by its position. Everything after it shuffles up.

USE IT WHEN

To take the item you have just dealt with off a queue.

EXAMPLE



Drop the first item; what was second becomes first.

● Used in G7 step 16 · See also **delete all of my list**

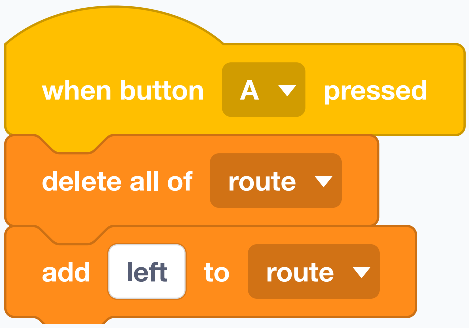
WHAT IT DOES

Empties a list completely.

USE IT WHEN

Before recording a new run. Without it, a second recording is added to the end of the first and the robot replays both.

EXAMPLE



Clear the slate, then start recording.

> An uploaded program does not start with empty lists if it saved anything before. Clear them yourself.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **add to my list**

insert [ ] at ( ) of [my list ▾]

WHAT IT DOES

Puts an item into the middle of a list at a position you choose, pushing the rest down.

USE IT WHEN

When order matters and the new thing does not belong at the end — inserting a correction into a recorded route.

EXAMPLE

```
when button [A ▾] pressed
insert [stop] at (1) of [route ▾]
```

A new first item; everything else moves down one.

See also **add to my list**

replace item ( ) of [my list ▾] with [ ]

WHAT IT DOES

Overwrites the item at one position with something else. The list stays the same length.

USE IT WHEN

To correct or update a stored value in place.

EXAMPLE

```
when button [A ▾] pressed
replace item (1) of [route ▾] with [right]
```

The first move was wrong; put the right one in its place.

See also **item of my list**

item ( ) of [my list ▾]

WHAT IT DOES

Reports one item out of a list, chosen by its position. The first item is 1, not 0.

USE IT WHEN

To play back what was collected — walking a stored route, reading a saved reading.

EXAMPLE

```
when button [A ▾] pressed
count with [i] from (1) to ((length of [route]) + (1)) by step (1) repeat
    print (item (i) of [route ▾])
```

Walk the whole list, one item at a time.

> The counting loop's end number is not included, so to reach the last item you count to the length PLUS ONE. Counting to the length alone silently misses the final item.

● Used in G7 step 15, G7 step 16, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **length of my list**, **count with from to by step repeat**

item # of [ ] in [my list ▾]

WHAT IT DOES

Reports the position of the first item matching what you give it, or 0 when the list does not contain it.

USE IT WHEN

To find where something is, not just whether it is there.

EXAMPLE

```
when button [A ▾] pressed
print (item # of [left] in [route ▾])
```

Where in the route is the first left turn?

See also **my list contains ?**

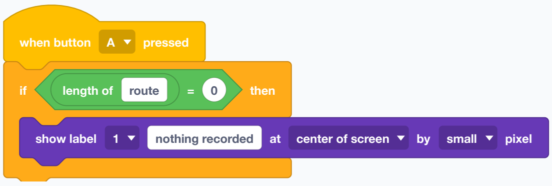
WHAT IT DOES

Reports how many items are in a list.

USE IT WHEN

To walk a list without knowing in advance how long it is, and to check whether anything was collected at all.

EXAMPLE



Notice an empty list before trying to play it back.

> There is a block in the Operators drawer worded almost the same, "length of", which counts the letters in a piece of text. Dropping a list name into THAT one measures the name: a list called route reports 5, always.

See also **length of**, **item of my list**

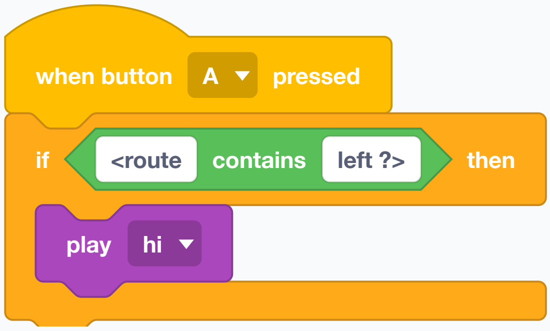
WHAT IT DOES

Reports true when the list holds the item, false when it does not.

USE IT WHEN

For checking membership — is this a word we know? has this place been visited?

EXAMPLE



Answer only if a left turn was recorded at some point.

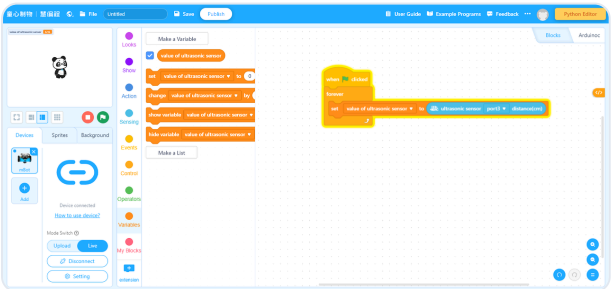
See also **item # of in my list**, **contains ?**

LIVE MODE ONLY

show variable [ ]

WHAT IT DOES

Shows the variable as a small value monitor on the stage inside the mBlock IDE on your computer. It does not appear on the CyberPi's physical screen.



The variable monitor appears in the upper-left corner of the mBlock IDE stage. Source: Makeblock.

USE IT WHEN

Live mode only, for debugging on the computer. An uploaded robot has no IDE stage, so use a Display "print" or "show label" block when you need to see the value on the CyberPi itself.

EXAMPLE

```
when green flag clicked
show variable [speed]
```

Watch a variable change in real time while testing at the desk.

See also **hide variable**

LIVE MODE ONLY

hide variable [ ]

WHAT IT DOES

Removes a variable's value monitor from the stage in the mBlock IDE. It does not affect the value stored in the variable.

USE IT WHEN

Live mode only, to tidy the IDE stage after debugging.

EXAMPLE

```
when green flag clicked
hide variable [speed]
```

Clear the stage of readouts.

See also **show variable**

show list [my list ▾]

WHAT IT DOES

Shows a list on the stage while the program runs.

USE IT WHEN

Live mode debugging, to watch a list fill up.

EXAMPLE

```
when green flag clicked
show list [route ▾]
```

Watch the route being recorded, item by item.

See also **hide list my list**

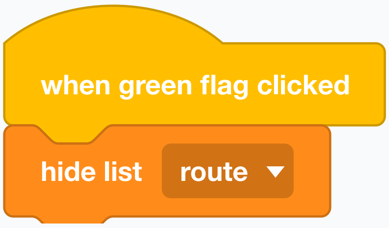
WHAT IT DOES

Takes a list's readout back off the stage.

USE IT WHEN

To clear the stage once you have finished watching.

EXAMPLE



Tidy the stage.

See also **show list my list**

My Blocks

Blocks you invent. Give a group of blocks a name and it becomes one block — which is how a long script turns into a short one that says what it means.

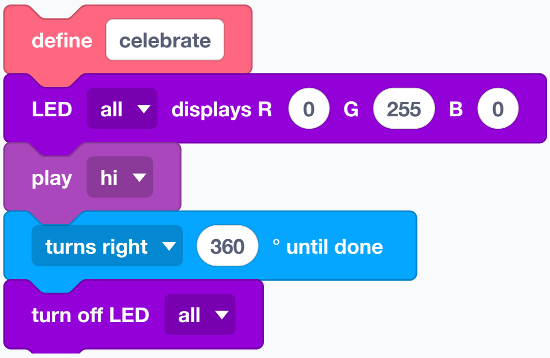
WHAT IT DOES

Starts a block of your own. You give it a name, and everything under the define hat becomes what that name does. The new block then appears in the My Blocks drawer for you to use like any other.

USE IT WHEN

As soon as a script gets long enough to lose your place in, or as soon as you want to do the same thing from two places. A well-named block turns six lines of driving into one line that says what the driving was FOR.

EXAMPLE



Four blocks, given the name "celebrate" — and from now on, one block.

> A define is not a script that runs. Nothing inside it happens until something else uses the block by name.

mBot2 Chassis

The two driving wheels use encoder motors: each motor has a built-in sensor that counts its rotation, so the robot can measure wheel speed and turning angle instead of only sending power and hoping. The encoder measures the motor's rotation, not actual distance across the floor, so wheel slip can still make a move inaccurate. EM1 and EM2 are the two numbered encoder-motor ports, not two different kinds of motor. On the standard mBot2, EM1 controls the left wheel and EM2 the right. On this course's Rover build, the cables are deliberately crossed: left is EM2 and right is EM1. The blocks whose labels say “until done” or “for … secs” hold the script while they run; the others start the wheels and let the script carry on, which is what you need to move and watch at once. RPM means revolutions per minute: 50 RPM asks a wheel to complete 50 full turns in one minute. A higher RPM means faster wheel rotation.

moves forward ▾ at (50) RPM for (1) secs

WHAT IT DOES

Drives the whole robot in one of four directions — forward, backward, turning left, turning right — at a speed you choose, for a number of seconds, and then stops the motors on its own.

Possible dropdown values

Menu shown as “moves forward”

`moves forward` `moves backward`
`turns left` `turns right`

USE IT WHEN

This is the simplest way to make one measured move. Reach for it when you want a definite movement and then the next thing: one side of a square, a nudge into position, a reverse out of a corner. Because it stops itself, you can never forget to switch the motors off.

EXAMPLE

```
when button A ▾ pressed
moves forward ▾ at (50) RPM for (1) secs
turns right ▾ (90) ° until done
```

One corner of a square: drive for a second, then turn a right angle.

> The script waits here. Nothing else in this script runs until the seconds are up, so you cannot watch a sensor while this block is driving.

See also **moves forward at RPM**, **moves forward cm until done**

moves forward ▾ at 50 RPM

WHAT IT DOES

Starts the robot moving and lets the script carry straight on. The wheels keep turning at that speed until some other block changes them or stops them.

Possible dropdown values

Menu shown as “moves forward”

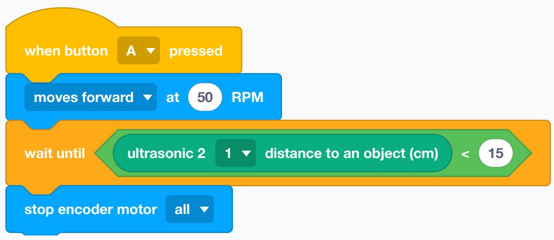
`moves forward` `moves backward`

`turns left` `turns right`

USE IT WHEN

Whenever the robot has to move and watch at the same time — drive forward while checking the distance sensor, follow a line, answer a joystick. This is the block for "keep going while I think about something else".

EXAMPLE



Creep forward and stop when something is closer than 15 cm.

> Nothing stops this by itself. If the script reaches its end the wheels keep turning, so finish with stop encoder motor, or set the speed to 0.

● Used in G7 step 9, G7 step 10, G7 step 11, G7 step 13, G7 step 14, G7 step 15 · See also **stop encoder motor (1) EM1**, **moves forward at RPM for secs**

moves forward ▾ 100 cm ▾ until done

WHAT IT DOES

Drives straight forward or backward by a measured distance in centimetres, and waits until the robot has covered it.

Possible dropdown values

Menu shown as “moves forward”

`moves forward` `moves backward`

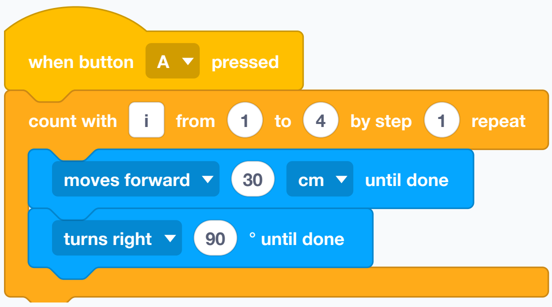
Menu shown as “cm”

`cm` `inch`

USE IT WHEN

When the distance matters more than the timing — a 30 cm side of a square, a 20 cm reverse. It counts wheel turns rather than seconds, so a low battery or a slightly grippier floor does not change how far the robot goes.

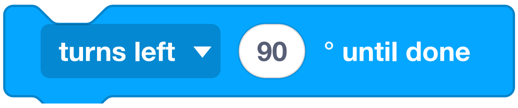
EXAMPLE



A 30 cm square: the same side and the same corner, four times.

> The centimetres are only as honest as the chassis settings. If your squares come out as rectangles, run calibrate chassis parameters once.

● Used in G7 step 4, G7 step 5, G7 step 6, G7 step 15, G7 step 16, G7 step 19 · See also **turns left ° until done**, **calibrate chassis parameters**



WHAT IT DOES

Spins the robot on the spot by a number of degrees — left or right — and waits until the turn is finished.

Possible dropdown values

Menu shown as “turns left”

`turns left` `turns right`

USE IT WHEN

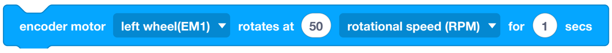
Any time you want a repeatable corner. 90° for a square, 120° for a triangle, 180° to come back the way you came, 360° for a celebration spin.

EXAMPLE

Turn around, then drive away from where you were pointing.

> Degrees depend on the wheels gripping. On a slippery floor the robot under-turns; the fix is a slower turn or a grippier surface, not a bigger number, because the wrong number breaks on the next floor.

Used in G7 step 5, G7 step 6, G7 step 13, G7 step 14, G7 step 15, G7 step 16 · See also **moves forward cm until done**, **calibrate chassis parameters**



WHAT IT DOES

Turns ONE wheel — left, right, or both — at a chosen speed for a number of seconds, then stops it. Speed is in RPM, and a negative number runs the wheel backwards.

Possible dropdown values

Menu shown as “left wheel(EM1)”

`left wheel(EM1)` `right wheel(EM2)` `all`

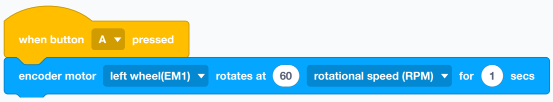
Menu shown as “rotational speed (RPM)”

`rotational speed (RPM)` `power (%)`

USE IT WHEN

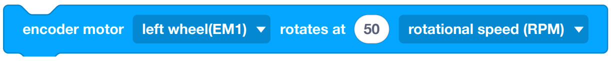
When the two wheels need different orders. Driving one wheel alone swings the robot around the other one, which is a much tighter turn than a spin on the spot.

EXAMPLE



The left wheel alone drives for a second, so the robot pivots around the right wheel.

See also **encoder motor left wheel(EM1) rotates at rotational speed (RPM)**, **encoder motor EM1 rotates at RPM**, **encoder motor EM2 rotates at RPM**



WHAT IT DOES

Sets one wheel turning at a speed and lets the script carry on. The wheel keeps going until something changes or stops it.

Possible dropdown values

Menu shown as “left wheel(EM1)”

`left wheel(EM1)` `right wheel(EM2)` `all`

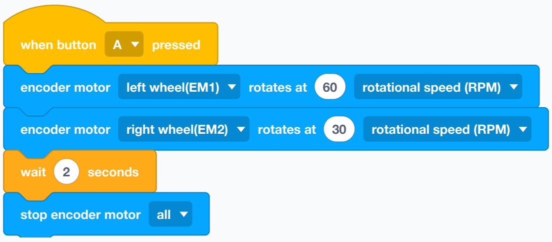
Menu shown as “rotational speed (RPM)”

`rotational speed (RPM)` `power (%)`

USE IT WHEN

The one-wheel version of "keep going while I think". Useful when only one side needs to change — nudging a curve wider while the other wheel holds its speed.

EXAMPLE



Two different wheel speeds make a long curve to the right.

> Like all the non-waiting movement blocks, this leaves the wheels running.

See also **encoder motor EM1 rotates at RPM, encoder motor EM2 rotates at RPM**, **stop encoder motor (1) EM1**

WHAT IT DOES

Turns one wheel by an exact number of degrees of its own rotation, and waits until it has finished. This is the wheel turning, not the robot turning.

Possible dropdown values

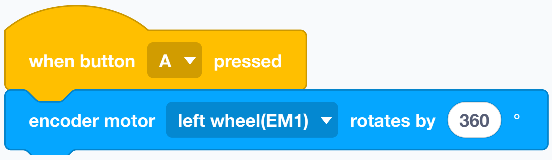
Menu shown as “left wheel(EM1)”

`left wheel(EM1)` `right wheel(EM2)` `all`

USE IT WHEN

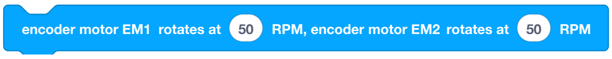
When you are driving something built onto the wheel shaft, or want a precise fraction of a wheel turn. For moving the robot a distance, the centimetre block is easier to reason about.

EXAMPLE



Exactly one full turn of the left wheel.

See also **encoder motor (1) EM1 rotated angle (°)**



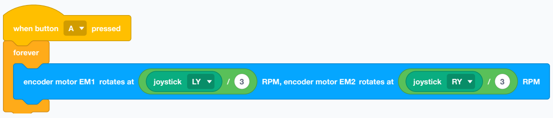
WHAT IT DOES

Gives both wheels a speed in one block: left wheel at one RPM, right wheel at another. The script carries straight on.

USE IT WHEN

This is the steering block. Equal speeds drive straight, a difference curves, and opposite signs spin on the spot. Every line follower and every joystick driver is built on it, because one block sets the whole state of the chassis.

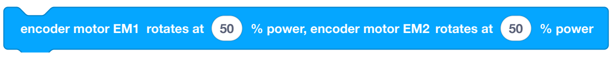
EXAMPLE



Tank steering: each stick drives the wheel on its own side, divided by 3 so full stick is a sensible speed rather than a bolt.

> On the standard mBot2, EM1 is the left wheel and EM2 the right. The Rover build used later in this course is wired the opposite way: left to EM2 and right to EM1. If the robot turns when both sides should drive forward, check which physical motor is connected to each port.

See also **encoder motor EM1 rotates at % power, encoder motor EM2 rotates at % power**, **joystick RX**



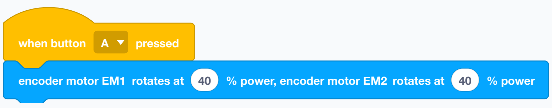
WHAT IT DOES

The same two-wheel block, but asking for a percentage of motor power instead of a target speed.

USE IT WHEN

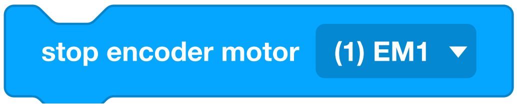
Rarely, in this course. Power says how hard to push; RPM says how fast to go and lets the robot push harder by itself when a wheel meets carpet or a slope. Prefer the RPM block unless you have a reason.

EXAMPLE



Both wheels at 40% power — the same effort, whatever the floor does to the actual speed.

See also **encoder motor EM1 rotates at RPM, encoder motor EM2 rotates at RPM**



WHAT IT DOES

Stops one wheel, or both, at once.

Possible dropdown values

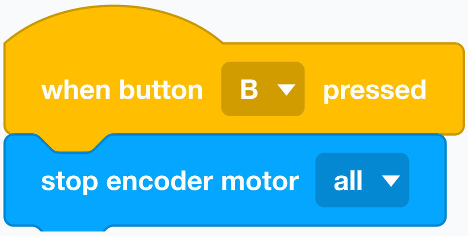
Menu shown as “(1) EM1”

`(1) EM1` `(2) EM2` `all`

USE IT WHEN

At the end of anything that used a non-waiting movement block. Choose "all" unless you really do mean one side.

EXAMPLE



A panic button: B stops the robot whatever it was doing.

> A stop hat like this only works if the script that is driving lets the CyberPi answer the button — in a forever loop, it does.

● Used in G7 step 9, G7 step 10, G7 step 11, G7 step 19, G7 step 20, G7 step 21 · See also **moves forward at RPM**, **stop all**

encoder motor (1) EM1 's rotational speed(RPM)

WHAT IT DOES

Reports how fast a wheel is actually turning right now, in RPM.

Possible dropdown values

Menu shown as “(1) EM1”

`(1) EM1` `(2) EM2`

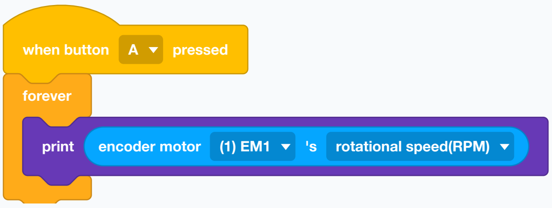
Menu shown as “rotational speed(RPM)”

`rotational speed(RPM)` `power (%)`

USE IT WHEN

For checking rather than commanding. Show it on the screen to see whether the robot is really reaching the speed you asked for — a wheel jammed against a wall reports far less than it was told.

EXAMPLE



Watch the left wheel's real speed on the CyberPi screen.

See also **encoder motor (1) EM1 rotated angle (°)**

encoder motor (1) EM1 rotated angle (°)

WHAT IT DOES

Reports how far a wheel has turned in degrees, counting up as it goes, since the last reset.

Possible dropdown values

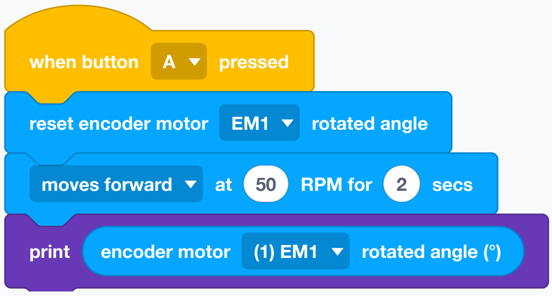
Menu shown as “(1) EM1”

`(1) EM1` `(2) EM2`

USE IT WHEN

When you want to measure a journey rather than time it. Reset the count, drive, and read the angle to know how far a wheel went.

EXAMPLE



Measure how many degrees the left wheel turned during a two-second drive.

> The count keeps growing until you reset it, so reset first or you will read this run plus every run before it.

● Used in G7 step 4, G7 step 5, G7 step 6, G7 step 19, G7 step 20, G7 step 21 · See also **reset encoder motor (1) EM1 rotated angle**

reset encoder motor (1) EM1 ▾ **rotated angle**

WHAT IT DOES

Sets a wheel's rotation count back to zero.

Possible dropdown values

Menu shown as "(1) EM1"

`(1) EM1` `(2) EM2` `all`

USE IT WHEN

Immediately before a movement you intend to measure.

EXAMPLE

when button A ▾ **pressed**
reset encoder motor EM1 ▾ **rotated angle**

Zero the left wheel's counter so the next reading starts from here.

● Used in G7 step 4, G7 step 5, G7 step 6, G7 step 19, G7 step 20, G7 step 21 · See also **encoder motor (1) EM1 rotated angle (°)**

encoder motor (1) EM1 ▾ enables ▾ **autolock**

WHAT IT DOES

Turns a wheel's autolock on or off. Locked, the motor actively holds its position and resists being pushed; unlocked, the wheel spins freely.

Possible dropdown values

Menu shown as "(1) EM1"

`(1) EM1` `(2) EM2` `all`

Menu shown as "enables"

`enables` `disenables`

USE IT WHEN

Lock when the robot must hold still on a slope or against a push. Unlock when you want to roll the robot by hand without fighting the motors.

EXAMPLE

when CyberPi starts up
encoder motor (1) EM1 ▾ enables ▾ **autolock**

Hold the left wheel in place from the moment the robot boots.

See also **stop encoder motor (1) EM1**

calibrate ▾ chassis parameters

WHAT IT DOES

Runs the chassis calibration routine, which teaches the robot its own wheel size and how far apart the wheels sit.

Possible dropdown values

Menu shown as “calibrate”

`calibrate` `reset`

USE IT WHEN

Once, after building the robot, and again if you change the wheels. Everything measured in centimetres or degrees is computed from these numbers, so a wrong setting makes every square a rectangle and every 90° corner a near-miss.

EXAMPLE

```
when button B ▾ pressed
calibrate ▾ chassis parameters
```

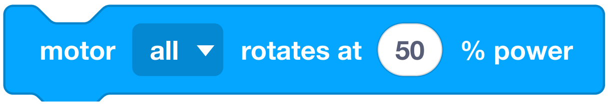
Put the robot on a clear floor and press B once.

> Give it space — it drives itself around while calibrating.

● Used in G8 step 3, G8 step 4, G8 step 5 · See also **moves forward cm until done**, **turns left ° until done**

mBot2 Extension Port

The four ports on the front for things you add yourself: servos, extra motors, LED strips, and raw pins for your own electronics. A pin is one small electrical contact in a connector. Different pins have different jobs: some supply power, one is ground, and others carry control signals or sensor readings. The pin order and voltage must match the part you connect; guessing can damage the part or the Shield. A digital read compares a signal with ground: near ground is low/false, while near the positive logic supply is high/true. A common pull-down switch is low while open and high when closed, but an active-low sensor deliberately reverses that meaning. An unconnected signal can float unpredictably and is not a valid reading. For a plain DC motor, M1 controls the motor connected to port M1, M2 controls the motor connected to port M2, and all sends the same command to both. M1/M2 are plain extension-motor ports; do not confuse them with EM1/EM2, the encoder-motor ports for the driving wheels.



WHAT IT DOES

Runs a plain DC motor on an extension port with a chosen amount of electrical drive: 0% is off, 50% is half power and 100% is full power. This controls how strongly the motor is driven, not an exact speed or angle. A negative number applies the power in the opposite direction.

Possible dropdown values

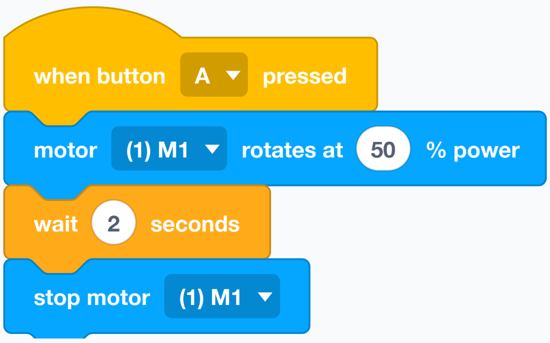
Menu shown as “all”

`all` `(1) M1` `(2) M2`

USE IT WHEN

For an added motor that just has to spin — a fan, a roller, a conveyor. These are not the wheel motors, which have their own blocks in the Chassis drawer. The same percentage can produce different speeds as the battery, load or friction changes.

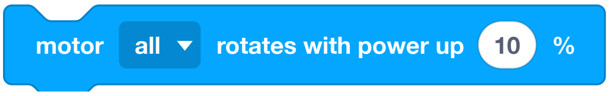
EXAMPLE



The extra motor receives half power for two seconds, then stops.

There is no angle slot because a plain DC motor has no encoder: it cannot measure how far it has turned or stop itself at a target angle. It keeps its last power command until another block changes or stops it. Use a servo to move to a particular position, or an encoder motor to measure rotation.

See also **stop motor all**, **encoder motor left wheel(EM1) rotates at rotational speed (RPM)**



WHAT IT DOES

Changes an extension motor's power by a step instead of setting it outright.

Possible dropdown values

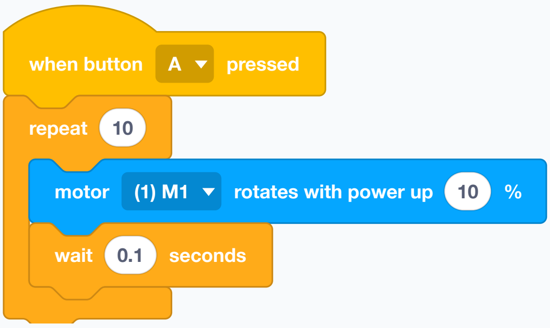
Menu shown as “all”

`all` `(1) M1` `(2) M2`

USE IT WHEN

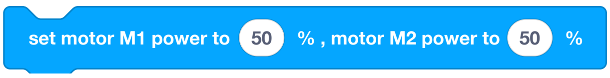
For ramping a motor up gently rather than starting it with a jolt.

EXAMPLE



A one-second run-up to full power.

See also **motor all rotates at % power**



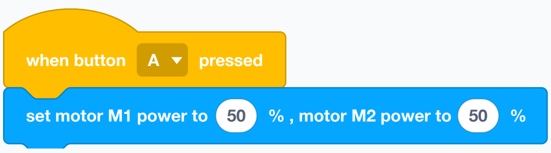
WHAT IT DOES

Sets both extension motors' power in one block.

USE IT WHEN

When two added motors work as a pair.

EXAMPLE



Both extra motors together.

See also **motor all rotates at % power**

WHAT IT DOES

Reports the power an extension motor is currently set to.

Possible dropdown values

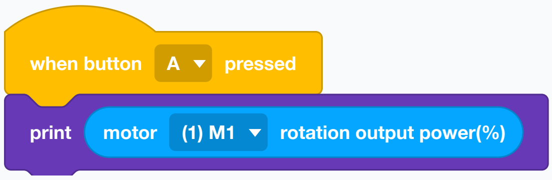
Menu shown as “(1) M1”

`(1) M1` `(2) M2`

USE IT WHEN

To display or to base the next change on.

EXAMPLE



Read the current power setting.

See also **motor all rotates at % power**

stop motor [all ▾]

WHAT IT DOES

Stops an extension motor, or all of them.

Possible dropdown values

Menu shown as “all”

`all` `(1) M1` `(2) M2`

USE IT WHEN

At the end of anything that started one. Like the wheel motors, they keep their last order.

EXAMPLE

```
when button [B ▾] pressed
stop motor [all ▾]
```

Everything on the extension ports stops.

See also **motor all rotates at % power**

set servo [all ▾] angle to (90) °

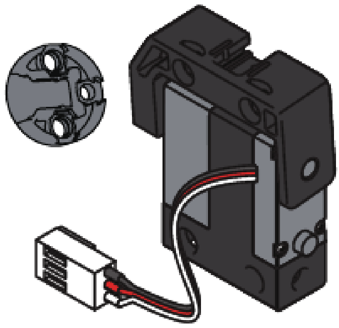
WHAT IT DOES

A servo is a small geared motor with a built-in position sensor and controller. Instead of being told how strongly to spin, it is given a target angle; it moves there, checks its position and applies force to hold that position. This block sends that angle to a servo on one of the four extension ports.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2` `(3) S3` `(4) S4`



The arm servo: a geared motor, position sensor and controller in one housing.

USE IT WHEN

This is the block for anything that must be in a POSITION rather than moving — a gripper open or shut, an arm raised or lowered, a sensor aimed. Unlike a plain DC motor, a standard servo moves through a limited angle range rather than spinning continuously.

EXAMPLE

```
when button [A ▾] pressed
set servo [(1) S1 ▾] angle to (90) °
wait (1) seconds
set servo [(1) S1 ▾] angle to (0) °
```

Open the gripper, pause, close it again.

> The servo will strain against a stop if you ask for an angle its build cannot reach. Find the two angles that

mean open and shut before writing the program around them.

● Used in G8 step 4, G8 step 5 · See also **servo (1) S1 current angle (°)**, **increase servo all angle by °**

WHAT IT DOES

Moves a servo by a number of degrees from where it is now, rather than to a fixed angle.

Possible dropdown values

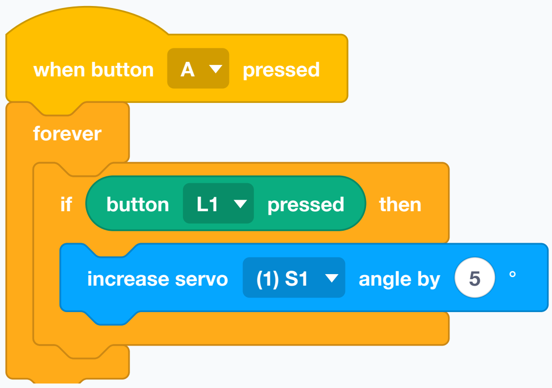
Menu shown as “all”

`all` `(1) S1` `(2) S2` `(3) S3` `(4) S4`

USE IT WHEN

For nudging under someone's control — a gripper that closes a little more each time a button is held.

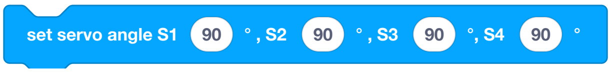
EXAMPLE



Hold the button and the arm creeps up.

Nudging has no memory of limits. Hold the button long enough and it will push past what the build allows.

● Used in G8 step 4, G8 step 5 · See also **set servo all angle to °**



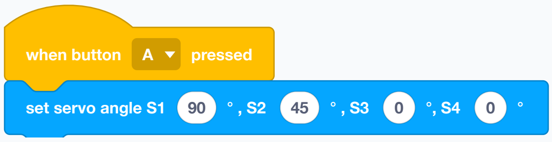
WHAT IT DOES

Sets all four servos to their own angles in one block.

USE IT WHEN

When several joints must move together — a whole arm to a pose, rather than one joint at a time.

EXAMPLE



A whole pose in one line.

See also **set servo all angle to °**

WHAT IT DOES

Reports the angle a servo is currently set to.

Possible dropdown values

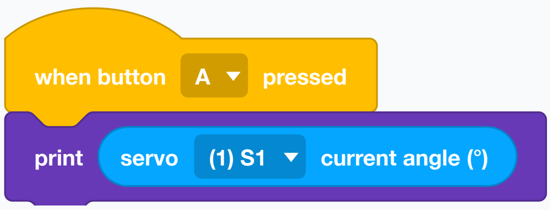
Menu shown as “(1) S1”

`(1) S1` `(2) S2` `(3) S3` `(4) S4`

USE IT WHEN

To show the position on screen, or to work out the next nudge from the current one.

EXAMPLE



Where is the arm right now?

● Used in G8 step 4, G8 step 5, G8 step 7, G8 step 8, G8 step 9, G8 step 10 · See also **set servo all angle to °**

servo all ▾ release angle

WHAT IT DOES

Lets a servo go limp, so it stops holding its angle and can be moved by hand.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2` `(3) S3` `(4) S4`

USE IT WHEN

To save power and to stop a servo buzzing while it fights a load it cannot hold. Also the way to pose an arm by hand before recording where it went.

EXAMPLE

```
when button B ▾ pressed
servo all ▾ release angle
```

Everything goes slack and can be moved by hand.

See also **set servo all angle to °**

servo all ▾ back to zero position

WHAT IT DOES

Treats the servo's current position as its new zero.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2` `(3) S3` `(4) S4`

USE IT WHEN

After assembling something, so the angles in your program are measured from where the build actually starts.

EXAMPLE

```
when CyberPi starts up
servo all ▾ back to zero position
```

Wherever the arm is now, call that zero.

See also **servo (1) S1 current angle (°)**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

`LED strip [all ▾] lights up [ ]`

WHAT IT DOES

Lights the first few LEDs of a strip, like a bar going up.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

USE IT WHEN

For a level meter — battery, speed, distance — where the number of lit lights IS the reading.

EXAMPLE

```
when button [A ▾] pressed
forever
    LED strip [all ▾] lights up (3)
```

Three lights on: a reading you can see from across the room.

See also **LED strip all LED all displays R: G: B:**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

`LED strip [all ▾] LED [all ▾] displays color [red]`

WHAT IT DOES

The same, with the colour picked from a swatch instead of mixed from numbers.

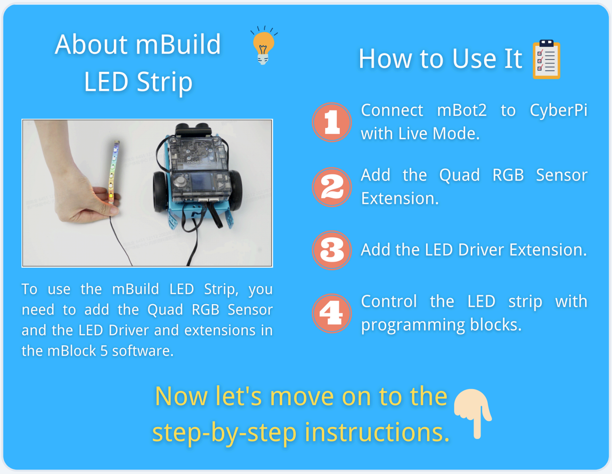
Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

Menu shown as “all”

`all` `1–36 (every whole number)`



A real mBuild LED strip illuminated beside the mBot2. This is separate from the five built-in LEDs on the CyberPi. Source: Makeblock.

USE IT WHEN

When you just want a colour.

EXAMPLE

```
when button [A ▾] pressed
LED strip [all ▾] LED [all ▾] displays color [ ]
```

Pick the colour on the block.

See also **LED strip all LED all displays R: G: B:**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

LED strip [all ▾] LED [all ▾] displays R: (255) G: (0) B: (0)

WHAT IT DOES

Lights one LED, or all of them, on a compatible three-wire programmable RGB strip connected through the mBot2 Shield or an mBuild LED Driver. This is not an ordinary single-colour strip or four-wire analogue RGB tape; the connector, voltage and control type must match. Makeblock sells the strip and driver as add-on hardware, but availability varies and the parts may need to be bought separately. They are not available in this classroom, so this block is outside the course and is included here only as palette reference.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

Menu shown as “all”

`all` `1–36 (every whole number)`

USE IT WHEN

When five lights on the board are not enough or not in the right place — a strip along an arm, or round the outside of a build.

EXAMPLE

```
when button [A ▾] pressed
LED strip [all ▾] LED [all ▾] displays R: (0) G: (255) B: (0)
```

The whole added strip goes green.

> This is the STRIP on an extension port, not the ring on the CyberPi. Those have their own blocks in the LED drawer. For every colour signal in this course, use the CyberPi's five built-in LEDs instead; no extra purchase is needed.

See also **LED strip all LED all displays color**, **LED all displays R G B**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

LED strip [all ▾] rolls (1) LEDs with cycle (15)

WHAT IT DOES

Shifts the pattern along the strip, wrapping round at the end.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

USE IT WHEN

To animate a strip — a dot running along an arm, a chase sequence.

EXAMPLE

```
when button [A ▾] pressed
forever
    LED strip [all ▾] rolls (1) LEDs with cycle ( )
    wait (0.1) seconds
```

The pattern travels along and comes back round.

See also **LED strip all lights up**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

WHAT IT DOES

Switches off one LED on the strip, or the whole strip.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

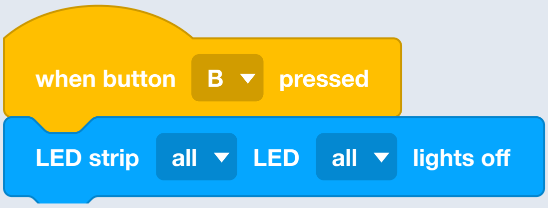
Menu shown as “all”

`all` `1–36 (every whole number)`

USE IT WHEN

At the end of a program.

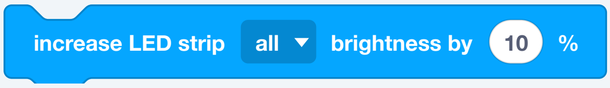
EXAMPLE



Strip off.

See also **LED strip all LED all displays R: G: B:**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE



WHAT IT DOES

Changes the strip's brightness by a step.

Possible dropdown values

Menu shown as “all”

`all` `(1) S1` `(2) S2`

USE IT WHEN

For fades.

EXAMPLE

The strip fades up.

See also **set LED strip all brightness to %**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

set LED strip [all ▾] brightness to (30) %

WHAT IT DOES

Sets the strip's brightness as a percentage.

Possible dropdown values

Menu shown as "all"

`all` `(1) S1` `(2) S2`

USE IT WHEN

Once at the start. A long strip at full brightness is both dazzling and a real drain on the battery.

EXAMPLE

when CyberPi starts up
set LED strip [all ▾] brightness to (40) %

Bright enough to see, kind to the battery.

See also **increase LED strip all brightness by %**

NOT IN THIS COURSE — HARDWARE UNAVAILABLE

LED strip [(1) S1 ▾] brightness(%)

WHAT IT DOES

Reports the strip's current brightness.

Possible dropdown values

Menu shown as "(1) S1"

`(1) S1` `(2) S2`

USE IT WHEN

To display the setting or continue a fade from it.

EXAMPLE

when button [A ▾] pressed
print (LED strip [(1) S1 ▾] brightness(%))

Read the brightness back.

See also **set LED strip all brightness to %**

pin (1) S1 on high level?

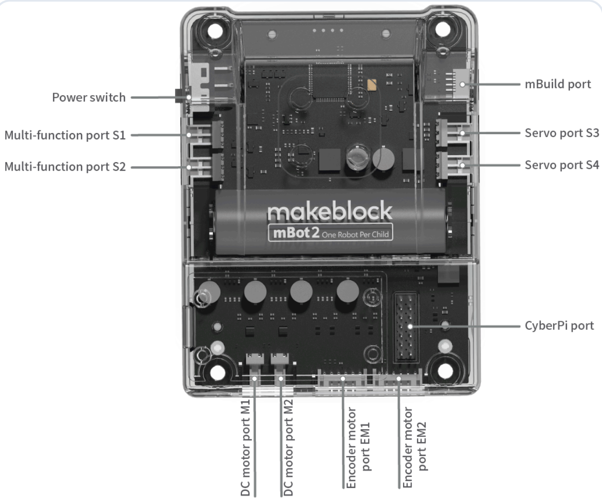
WHAT IT DOES

Reads the voltage arriving on the signal contact inside connector S1 or S2. Each connector also has separate power and ground contacts, but this block does not read those. It converts the signal voltage into a yes-or-no answer: a high level reports true and a low level reports false. The connector is not a sensor by itself; an external switch, sensor or circuit must supply the signal.

Possible dropdown values

Menu shown as “(1) S1”

`(1) S1` `(2) S2`



S1 and S2 are two multifunction connectors on the mBot2 Shield. “(1) S1” selects connector S1 and “(2) S2” selects connector S2; (1) and (2) are menu identifiers, not physical pin numbers. Source: Makeblock.

USE IT WHEN

With custom electronics that provide a digital high-or-low output, inside an if or a wait. Connect the part's power, ground and signal correctly before reading it; this course does not ask students to build that circuit.

EXAMPLE

```
when button A pressed
wait until pin (1) S1 on high level?
play hi
```

Signal flow: external switch or sensor → S1 signal contact → high/low answer. With nothing connected, the input may be unstable and should not be trusted.

> In a common pull-down switch circuit, an open switch leaves the signal near ground, so it is low/false; closing the switch connects the signal to the port's positive supply, so it is high/true. A digital sensor can do the same: for example, high when it detects an object and low when it does not. Some devices are active-low and deliberately reverse those meanings, so always check the device's wiring or documentation. An unconnected signal can float between states and is not a meaningful reading.

See also **digital read pin (1) S1**

digital read pin (1) S1

WHAT IT DOES

Reports whether a pin is currently reading on or off.

Possible dropdown values

Menu shown as “(1) S1”

`(1) S1` `(2) S2`

USE IT WHEN

For a switch or button of your own wired to a port.

EXAMPLE

```
when button A pressed
forever
    print digital read pin (1) S1
```

Watch the state of a home-made switch.

See also **pin (1) S1 on high level?**

voltage read pin (1) S1 ▾ (V)

WHAT IT DOES

Reports the actual voltage on a pin, rather than just on or off.

Possible dropdown values

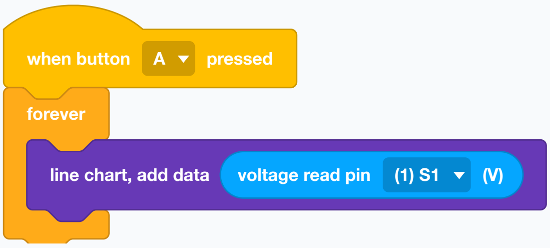
Menu shown as “(1) S1”

`(1) S1` `(2) S2`

USE IT WHEN

For sensors that answer with a voltage instead of a switch — a light-dependent resistor, a potentiometer, a home-made divider.

EXAMPLE



A home-made sensor, drawn as a graph.

See also **digital read pin (1) S1**

Mini-course: digital pins, voltage, high and low

Start with the connector. S1 and S2 are multifunction ports on the mBot2 Shield. Each connector contains separate electrical contacts for power, ground and a signal. The raw pin blocks read or control the *signal* contact—not the whole connector.

Voltage needs a reference

Voltage is a difference between two electrical points. Here, the Shield compares the signal with ground. Ground is the reference called 0 V.

Signal state	Electrical situation	Digital reading
Low	Signal is deliberately held near ground.	false / 0
High	Signal is deliberately driven near the positive logic supply.	true / 1
Floating	Signal is connected to neither ground nor a defined high level.	Unreliable; it may jump between states.

```
ground → signal = low
power → signal = high
nothing → signal = floating, not “high”
```

A switch example

In a common **pull-down** circuit, a resistor holds the signal low while the switch is open. Closing the switch connects the signal to the positive supply, making it high. The resistor prevents the open signal from floating.

Sensors may reverse the meaning

An active-high sensor might output high when it detects an object. An **active-low** sensor outputs low when it detects one. Neither convention is universally correct: check that sensor’s documentation.

Writing sends a signal out

Digital write drives the signal low (0) or high (1). It is like a software-controlled electrical switch: there is no middle setting. **Digital read** observes a signal coming in; digital write creates an output signal.

“Analog write” is rapid digital switching

On these blocks, analog write means **PWM** (pulse-width modulation). The output remains digital, but switches high and low quickly. The duty cycle says how much of each cycle is high; the frequency says how many cycles happen each second.

Duty cycle	Output pattern	Typical effect
0%	Always low	Off
50%	High half the time	About half average power
100%	Always high	Fully on

Two real output cases

Digital warning lamp: a distance test can write 1 to switch an external warning LED on when an object is too close, then write 0 when the path is clear. The output has two intentional states: on or off.

PWM night light: analog write can use a low duty cycle for a dim night mode and 100% for a bright mode. The LED averages the fast pulses into visible brightness. Motors, relays and high-current lights require a proper driver circuit; never power them directly from a signal pin.

Course boundary and safety

These raw pin blocks are reference material; this course does not require custom S1/S2 circuits. Never guess a connector's pin order or voltage. Wrong wiring can damage the external part or the Shield.

digital write (1) to pin (all ▾)

WHAT IT DOES

Drives the selected S1/S2 signal contact to one of two output levels. Writing 0 holds it low, near ground; writing 1 drives it high, near the positive logic supply. Unlike digital read, which observes an incoming signal, digital write sends a signal out.

Possible dropdown values

Menu shown as "all"

`all` `(1) S1` `(2) S2`

USE IT WHEN

For simple electronics of your own: a buzzer, a relay, a single LED wired to the port.

EXAMPLE

```
when button (A ▾) pressed
digital write [ ] to pin ((...) S1 1 ▾)
when button (B ▾) pressed
digital write [ ] to pin ((...) S1 0 ▾)
```

Real case: an external warning LED turns on with A and off with B. The LED needs the correct series resistor (and a driver if its current is too high).

> A signal pin can control a compatible low-current circuit; it is not a power outlet for a motor or other heavy load. Check the part's voltage, current and wiring requirements before connecting it.

See also **digital read pin (1) S1**

analog write to pin (all ▾) , duty cycle (50) %, frequency (2000 ▾) Hz

WHAT IT DOES

"Analog write" here is PWM, not a continuously adjustable analog voltage. The pin still outputs only high or low, but switches rapidly between them. Duty cycle is the percentage of each cycle spent high; frequency is how many cycles occur per second. At 50% duty cycle, the signal is high half the time and low half the time.

Possible dropdown values

Menu shown as "all"

`all` `(1) S1` `(2) S2`

Menu shown as "2000"

`2000` `1000` `500` `200` `100` `50` `20` `10` `5` `2`

USE IT WHEN

To dim a plain LED or run a small motor at part speed from a pin that can only be on or off. Half-on, thousands of times a second, looks like half brightness.

EXAMPLE

```
when button (A ▾) pressed
analog write to pin ((1) S1 ▾) , duty cycle (20) %, frequency (2000 ▾) Hz
when button (B ▾) pressed
analog write to pin ((1) S1 ▾) , duty cycle (100) %, frequency (2000 ▾) Hz
```

Real case: a night-light LED is dim at 20% after A and fully bright at 100% after B. Use the correct resistor and output-driver circuit for the load.

See also **digital write to pin all**

Quad RGB Sensor

> The four colour probes underneath, which read the floor. They can answer a yes-or-no question about one probe, or report how far off a line the whole robot has drifted.

Guide: choosing values in Quad RGB Sensor blocks

Read each long block as a sequence of choices: **which sensor module → which probe(s) → what to detect or measure → which result to test.**

1. Sensor number: 1–8

The first number identifies a Quad RGB Sensor in an mBuild chain. It does not select one of its four probes. This course has one Quad RGB Sensor, so leave it at **1**; 2–8 are for additional modules.

2. Probe: R2, R1, L1 or L2

Left and right are from the robot's point of view as it drives forward. The menu numbers count from the outer-right probe toward the left:

(4) L2 **(3) L1** **(2) R1** **(1) R2**

The middle probes L1 and R1 normally straddle the line. L2 and R2 help detect larger drift, wide bands and junctions.

3. Detection choices

Choice	Meaning
line / background	The **line** is the path the robot follows; the **background** is the surrounding track surface. With black tape on white paper, black is the line and white is the background. These are surface categories learned during calibration, not colours named in the block.
white, red, yellow, green, cyan, blue, purple, black	A yes-or-no test for a built-in colour.
self-defined color	The custom RGB colour and tolerance previously taught with the "define color" block.

Calibration defines "line" and "background"

Calibration is an action performed before line detection. Start it by double-pressing the button on the Quad RGB Sensor, or by running the **"quad rgb sensor 1 performs calibration"** command block. While it is calibrating, move the sensor across both the line and the surrounding background. It measures and stores the difference; later blocks use those stored values whenever they ask about **line** or **background**.

Calibrate again when the track, room lighting, sensor height or fill-light setting changes. Without a suitable calibration, the sensor may confuse the two surfaces.

4. Measurement choices

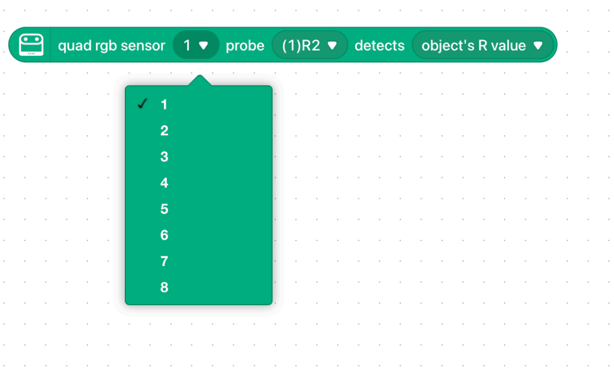
Choice	Returned value
object's R / G / B value	Red, green or blue intensity from 0 to 255.
object's grayscale	Reflected-light strength from 0 to 100, independent of a colour name.
ambient light intensity	Light reaching the probe, from 0 to 100. The fill light contributes unless turned off.
color	The measured colour as hexadecimal RGB, from `0x000000` to `0xFFFFFF`.

5. Status codes

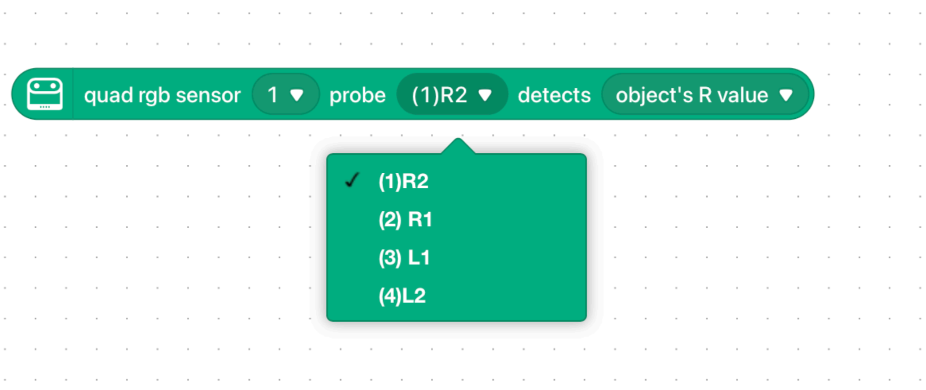
Each digit belongs to one probe. **1** means it detects the condition selected in the block; **0** means it does not. The middle-pair order is **L1 R1**, so `10` means L1 yes and R1 no. The four-probe order is **L2 L1 R1 R2**, so `0110` means the two middle probes detect it and the outer probes do not. Two probes have 4 possible patterns; four probes have all 16 patterns from `0000` to `1111`.

6. Fill-light colour

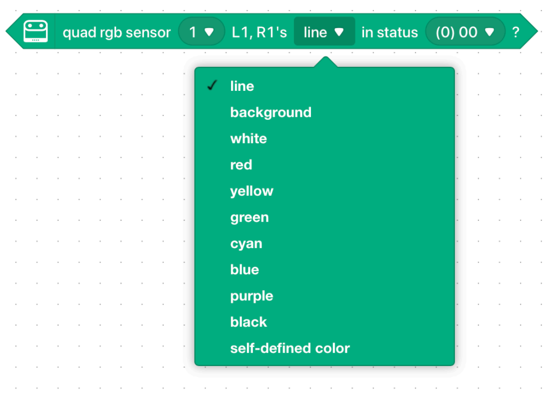
The sensor shines red, green or blue light onto the floor before reading the reflection. This is illumination, not a detected colour. Changing it can improve contrast, but also changes readings—calibrate under the same light and surface conditions in which the robot will run.



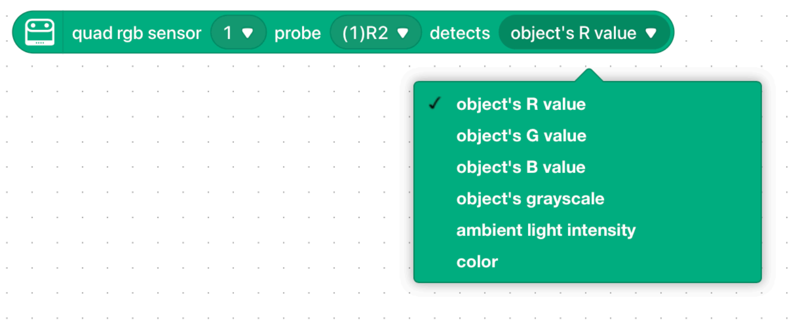
Sensor number: choose the module; this course uses 1.



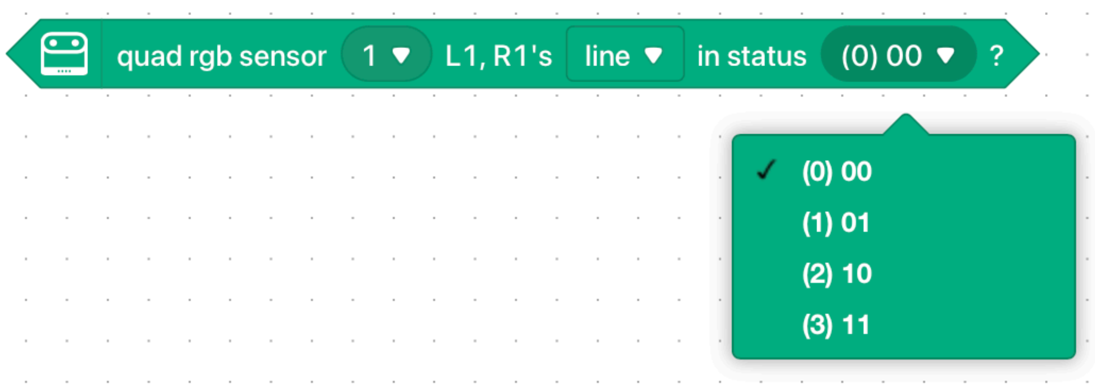
Probe: choose one of the four sensing positions.



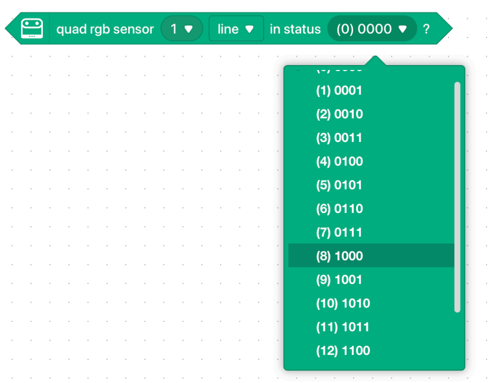
Detection target: ask about line/background or a colour.



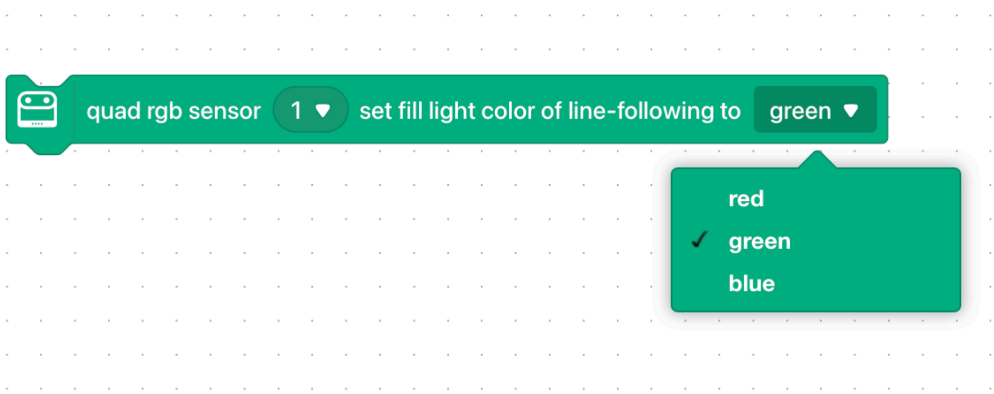
Measurement: report a value instead of yes or no.



Middle pair: digits are L1 then R1.



All four: digits are L2, L1, R1, R2.



Fill light: choose the light shone onto the surface.

quad rgb sensor 1 L1, R1's line in status (0) 00 ?

WHAT IT DOES

Asks whether the two middle probes together are in one exact pattern — both miss the selected condition, left only, right only, or both detect it. The digits are ordered L1 then R1; 10 means L1 yes and R1 no.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “line”

`line` `background` `white` `red` `yellow` `green` `cyan` `blue` `purple` `black` `self-defined color`

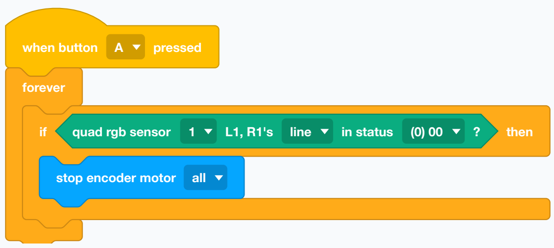
Menu shown as “(0) 00”

`(0) 00` `(1) 01` `(2) 10` `(3) 11`

USE IT WHEN

When the pair of readings matters as a pair. "Both on the line" means straight ahead; "left only" means drifting right.

EXAMPLE



Both middle probes have lost the line, so stop rather than guess.

See also **quad rgb sensor 1 L1, R1's line status (0~3)**

quad rgb sensor 1 L1, R1's line status (0~3)

WHAT IT DOES

Reports the two middle probes as a single number from 0 to 3 instead of a yes-or-no.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

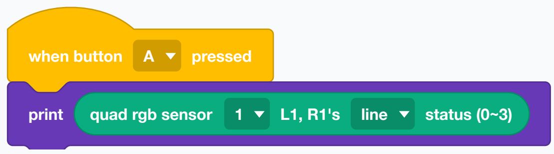
Menu shown as “line”

`line` `background` `white` `red` `yellow` `green` `cyan` `blue` `purple` `black` `self-defined color`

USE IT WHEN

When you would rather compare a number than write out four separate questions.

EXAMPLE



Watch the pair of middle probes as one changing number.

See also **quad rgb sensor 1 L1, R1's line in status (0) 00 ?**

quad rgb sensor 1 line in status (0) 0000

WHAT IT DOES

Asks whether all FOUR probes are in one exact pattern, written in physical left-to-right order: L2, L1, R1, R2. Each 1 means that probe detects the selected condition; each 0 means it does not.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “line”

`line` `background` `white` `red` `yellow` `green` `cyan` `blue` `purple` `black` `self-defined color`

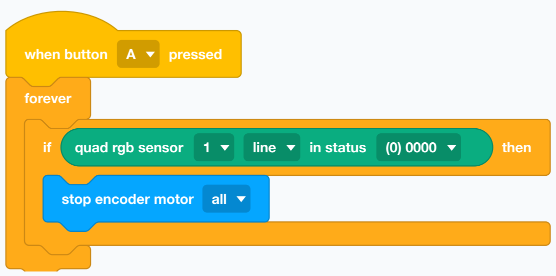
Menu shown as “(0) 0000”

`(0) 0000` `(1) 0001` `(2) 0010` `(3) 0011` `(4) 0100` `(5) 0101` `(6) 0110` `(7) 0111` `(8) 1000` `(9) 1001` `(10) 1010` `(11) 1011` `(12) 1100` `(13) 1101` `(14) 1110` `(15) 1111`

USE IT WHEN

For reading marks on the floor rather than following a line. A wide band under all four probes is a different pattern from a thin line under the middle two, so the robot can tell a junction from a track.

EXAMPLE



No probe can see the line at all — the robot has come right off the track.

See also **quad rgb sensor 1 line status (0~15)**

quad rgb sensor 1 line status (0~15)

WHAT IT DOES

Reports all four probes as one number from 0 to 15.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

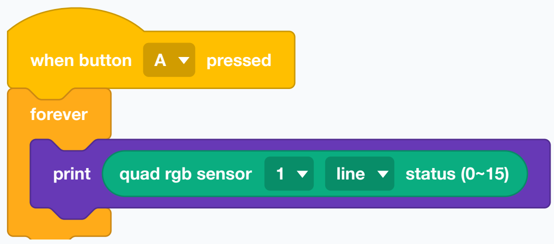
Menu shown as “line”

`line` `background` `white` `red` `yellow` `green` `cyan` `blue` `purple` `black` `self-defined color`

USE IT WHEN

Same idea as the four-digit pattern, as a number you can compare or store. Sixteen values cover every combination of four probes.

EXAMPLE



The whole sensor as one number, changing as the robot crosses the track.

See also **quad rgb sensor 1 line in status (0) 0000**

quad rgb sensor 1 probe (1) R2 detects line ?

WHAT IT DOES

Asks one of the four probes a yes-or-no question: are you over the line? over the background? over a particular colour?

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “(1) R2”

`(1) R2` `(2) R1` `(3) L1` `(4) L2` `random`

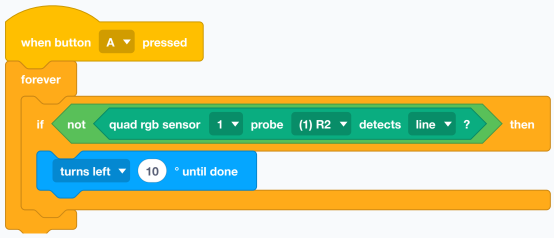
Menu shown as “line”

`line` `background color` `white` `red` `green` `blue` `yellow` `cyan` `purple` `black`

USE IT WHEN

When you want a decision rather than a measurement — "have we reached the red patch", "has the outer probe fallen off the line". Easier to reason about than the deviation number when the answer is genuinely yes or no.

EXAMPLE



Hunt left whenever the right-hand probe loses the line.

> The probes are numbered from the right: R2, R1, L1, L2. Facing the same way as the robot, R2 is the outer right one.

Used in G8 step 14, G8 step 15 · See also **quad rgb sensor 1 deviation (-100~100)**

quad rgb sensor 1 probe (1)R2 detects object's R value

WHAT IT DOES

Reports one measurement from the selected probe. “object's R value”, G value and B value report the strength of reflected red, green and blue light from 0 to 255; a larger number means more of that colour channel. “object's grayscale” reports reflected-light strength from 0 to 100 without naming a colour: dark, weakly reflecting surfaces tend toward the low end and bright, strongly reflecting surfaces toward the high end. “ambient light intensity” reports the light reaching the probe from 0 to 100; the sensor's fill light contributes unless it is turned off. “color” reports the measured colour as one hexadecimal RGB value from 0x000000 (black) to 0xFFFFFF (white), rather than returning a colour name.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “(1)R2”

`(1)R2` `(2) R1` `(3) L1` `(4)L2`

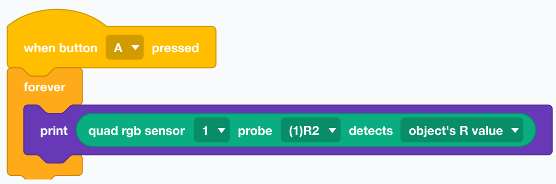
Menu shown as “object's R value”

`object's R value` `object's G value` `object's B value` `object's grayscale` `ambient light intensity` `color`

USE IT WHEN

For telling colours apart rather than following a line, and for finding out why a line follower is failing. Printing the grey value over black tape and over white paper shows you the gap the sensor has to work with.

EXAMPLE



Watch the raw red reading change as different colours pass under the probe.

> Raw values depend on the surface, sensor height, room lighting and fill-light setting. Compare readings under the same conditions rather than treating one number as a universal definition of a colour.

See also **quad rgb sensor 1 define color to R G B with tolerance**

quad rgb sensor 1 ▾ deviation (-100~100)

WHAT IT DOES

Reports how far the robot has drifted off the line, as a number from -100 to +100. Zero means dead centre; the sign says which way it has wandered.

Possible dropdown values

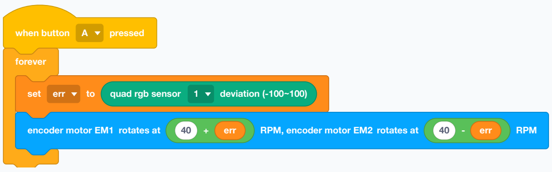
Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

This is the line-following block. One number carries both how far off and which side, which is exactly what a steering correction needs — no chain of ifs, just arithmetic.

EXAMPLE



Drift left and the left wheel speeds up while the right slows, which steers back. The bigger the drift, the harder the correction.

> If the robot wobbles violently, the correction is too strong — divide the error before using it. If it drifts off corners, it is too weak.

● Used in G7 step 20, G7 step 21 · See also **quad rgb sensor 1 probe (1) R2 detects line ?**, **abs of**

quad rgb sensor 1 ▾ set fill light color of line-following to red ▾

WHAT IT DOES

Sets the colour of the sensor's own lamp — the light it shines on the floor to read by.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

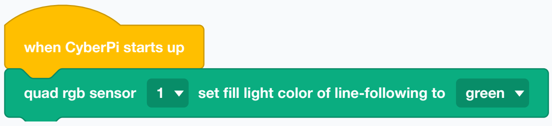
Menu shown as “red”

`red` `green` `blue`

USE IT WHEN

When the track colour and the lamp colour fight each other. Red tape under a red lamp looks the same as white paper.

EXAMPLE



A green lamp makes red tape stand out sharply.

See also **quad rgb sensor 1 turns off fill light**

quad rgb sensor 1 turns off fill light

WHAT IT DOES

Switches the sensor's lamp off.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

To read the room's own light rather than the robot's, or to save power.

EXAMPLE

when button B pressed
quad rgb sensor 1 turns off fill light

Lamp off — now the probes see only what the room lights show them.

> Line following with the lamp off is unreliable, because the readings then change with every shadow.

See also **quad rgb sensor 1 set fill light color of line-following to red**

quad rgb sensor 1 define color to R 0 G 0 B 0 with tolerance 50

WHAT IT DOES

Teaches the sensor a colour: you give it red, green and blue values plus a tolerance for how close a match has to be.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

When the surface you care about is not one of the built-in colours, or when the classroom's paper is a shade the sensor keeps mistaking.

EXAMPLE

when CyberPi starts up
quad rgb sensor 1 define color to R 200 G 30 B 30 with tolerance 50

Teach it your particular red, with room for the light to change.

> A big tolerance matches too much, a small one matches nothing. Start around 50 and adjust while watching the raw values.

See also **quad rgb sensor 1 probe (1)R2 detects object's R value**

quad rgb sensor 1 performs calibration

WHAT IT DOES

Runs the sensor's own calibration. The line is the path the robot follows; the background is the surrounding track surface. Calibration measures both and stores their difference, so later blocks can decide which one each probe sees. You can also start this action by double-pressing the button on the Quad RGB Sensor.

Possible dropdown values

Menu shown as "1"

`1–8 (every whole number)`

USE IT WHEN

Once at the start of a session, and again whenever the room or the track changes. Sunlight through a window in the afternoon is not the light the sensor was calibrated in that morning.

EXAMPLE

```
when button B pressed
quad rgb sensor 1 performs calibration
```

Press B, then move the sensor across both the line and the background while it learns the two surfaces.

> Calibrate over the real track, not over the desk. Repeat it if the track, room lighting, sensor height or fill-light setting changes.

Used in G7 step 20, G7 step 21, G7 step 24 · See also **quad rgb sensor 1 deviation (-100~100)**

sets color detection mode to Standard

WHAT IT DOES

Switches the sensor between Standard and Color enhanced detection.

Possible dropdown values

Menu shown as "Standard"

`Standard` `Color enhanced`

USE IT WHEN

Enhanced mode when you are separating similar colours; standard mode for plain line following, where it is faster.

EXAMPLE

```
when CyberPi starts up
sets color detection mode to Color enhanced
```

Trade a little speed for a better eye for colour.

See also **quad rgb sensor 1 define color to R G B with tolerance**

Ultrasonic Sensor 2

The forward-looking distance sensor, with two ring lights for eyes. It belongs to the standard mBot2 — the Rover build does not have one.

ultrasonic 2 1 ▾ distance to an object (cm)

WHAT IT DOES

Reports how far away the nearest thing in front of the sensor is, in centimetres.

Possible dropdown values

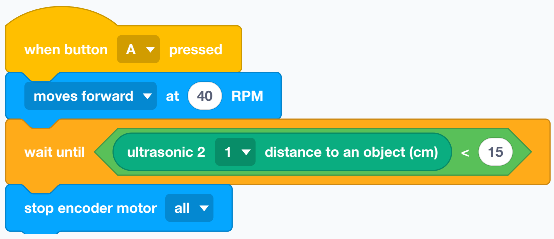
Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

Any time the robot must not hit something, or must stop a set distance from it. It is the robot's only forward-looking sense of space.

EXAMPLE



Drive up to a wall and stop a hand's width from it.

This sensor belongs to the standard mBot2. The Rover build does not have one, which is why the Grade 8 course never uses it.

Used in G7 step 8, G7 step 9, G7 step 10, G7 step 11, G7 step 13, G7 step 14 · See also **ultrasonic 2 1 out of range** , **<**

ultrasonic 2 1 ▾ out of range

WHAT IT DOES

Reports true when there is nothing in range to measure.

Possible dropdown values

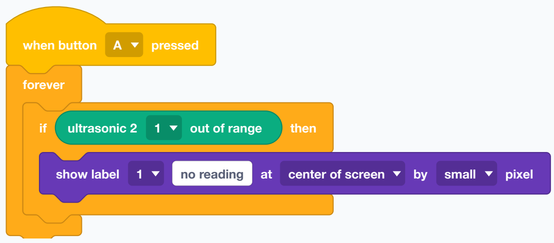
Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

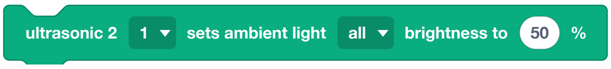
To tell "far away" apart from "no reading at all". Soft or angled surfaces bounce the sound away instead of back, and an empty corridor gives the same silence as a cushion.

EXAMPLE



Say so, rather than acting on a distance that was never measured.

See also **ultrasonic 2 1 distance to an object (cm)**



WHAT IT DOES

Sets how bright the sensor's ring lights are, as a percentage.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

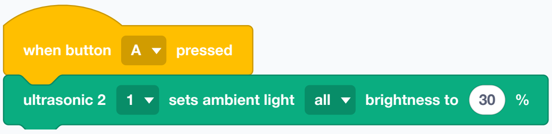
Menu shown as “all”

`all` `1–8 (every whole number)`

USE IT WHEN

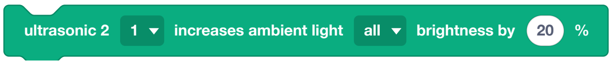
To dim the eyes in a dark room, or use brightness itself as a signal — brighter as an obstacle gets nearer.

EXAMPLE



Both rings at a third brightness.

See also **ultrasonic 2 1 increases ambient light all brightness by %**



WHAT IT DOES

Raises or lowers the ring brightness by a step, rather than setting it outright.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

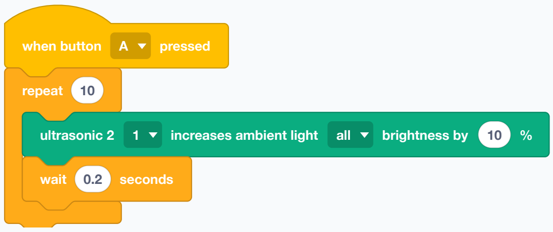
Menu shown as “all”

`all` `1–8 (every whole number)`

USE IT WHEN

For a fade, or for a brightness that responds to something without your having to track the current level.

EXAMPLE



The eyes fade up over two seconds.

See also **ultrasonic 2 1 sets ambient light all brightness to %**

ultrasonic 2 (1 ▾) ambient light (1 ▾) brightness

WHAT IT DOES

Reports how bright one of the rings is set to right now.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “1”

`1–8 (every whole number)`

USE IT WHEN

When a fade needs to know where it has got to.

EXAMPLE

when button (A ▾) pressed
print (ultrasonic 2 (1 ▾) ambient light (1 ▾) brightness)

Read the current brightness back off the sensor.

See also **ultrasonic 2 1 sets ambient light all brightness to %**

ultrasonic 2 (1 ▾) turns off ambient light (all ▾)

WHAT IT DOES

Turns the sensor's ring lights off.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “all”

`all` `1–8 (every whole number)`

USE IT WHEN

At the end of a program, or when the eyes are distracting from something else.

EXAMPLE

when button (B ▾) pressed
ultrasonic 2 (1 ▾) turns off ambient light (all ▾)

Eyes off.

See also **ultrasonic 2 1 displays emotion standby**

ultrasonic 2 [1 ▾] displays emotion [standby ▾]

WHAT IT DOES

Plays one of the sensor's built-in faces on its two ring lights — standby, happy, dizzy, raised eyebrows, thinking.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “standby”

`standby` `happy` `dizzy` `raise eyebrows` `think`

USE IT WHEN

To give the robot an expression the whole room can read from a distance. The eyes are on the front of the robot, which is where people look.

EXAMPLE

when button [A ▾] pressed
ultrasonic 2 [1 ▾] displays emotion [happy ▾]

The robot's face lights up.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G7 step 9, G7 step 10 · See also **ultrasonic 2 1 sets ambient light all brightness to %**

Bluetooth controller

The gamepad. These blocks come from the Bluetooth Controller EXTENSION and work in Upload mode; the identical-looking blocks from the bundled controller device do not.

joystick RX

WHAT IT DOES

Reports how far one stick on the Bluetooth gamepad is pushed, on one axis, as a number either side of zero. LX and LY are the left stick, RX and RY the right.

Possible dropdown values

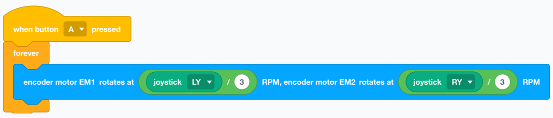
Menu shown as "RX"

`RX` `RY` `LX` `LY`

USE IT WHEN

For driving by hand. Feeding the two vertical axes to the two wheels gives tank steering, which is the arrangement this course uses.

EXAMPLE



Each stick drives the wheel on its own side.

> These blocks only work from the Bluetooth Controller EXTENSION, and only in Upload mode. The similar-looking blocks that come with the bundled controller DEVICE carry no code for the robot, so they upload and do nothing.

● Used in G8 step 12, G8 step 13, G8 step 14, G8 step 15 · See also **button 1 pressed**, **encoder motor EM1 rotates at RPM, encoder motor EM2 rotates at RPM**

button 1 pressed

WHAT IT DOES

Reports true while a named button on the gamepad is held — the numbered buttons, the arrows, the shoulder buttons, the thumbsticks.

Possible dropdown values

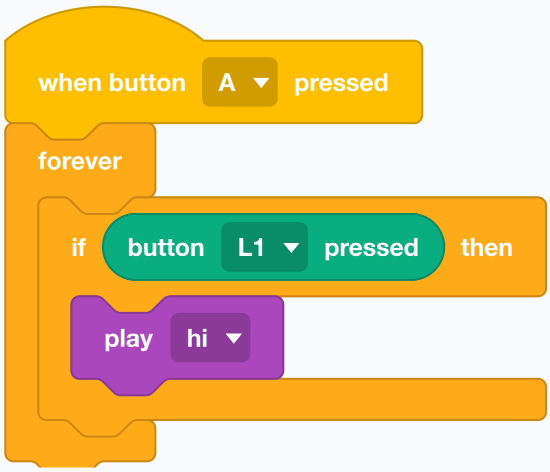
Menu shown as "1"

`1` `2` `3` `4` `←` `↑` `→` `↓` `L1` `R1` `L2` `R2` `Left Thumb` `Right Thumb` `+` `≡`

USE IT WHEN

For everything that is not steering: a gripper, a horn, a light, a mode change. Inside a forever loop it reads as "while this is held".

EXAMPLE



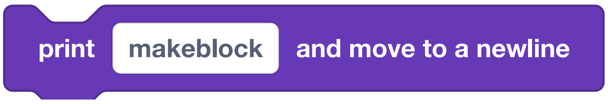
The shoulder button sounds the horn for as long as it is held.

> Upload mode, and the extension rather than the device — the same warning as the joystick block.

See also **joystick RX**

Display

The colour screen. Labels stay in fixed slots and update in place; printing scrolls; charts and tables are for numbers that change.



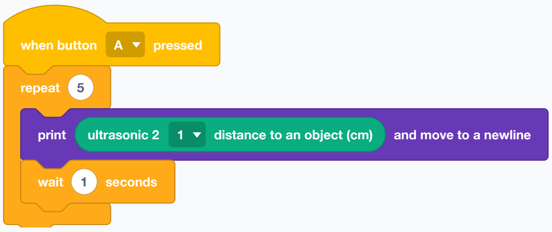
WHAT IT DOES

Prints text and then moves to the next line, so the next print starts underneath.

USE IT WHEN

For a running log rather than a fixed readout — a list of readings as they arrive, a trace of what the program did.

EXAMPLE



Five readings, one per line, one per second.

The screen scrolls once it is full, so old lines are lost. For something that must stay visible, use a label.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G8 step 13, G8 step 14 · See also **print**, **clear screen**

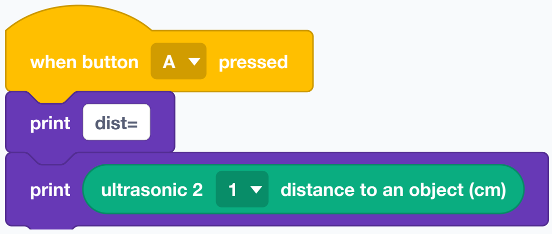
WHAT IT DOES

Adds text to the screen where the last print left off, without starting a new line.

USE IT WHEN

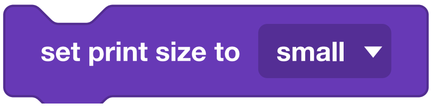
To build one line out of several pieces — a name, then its value.

EXAMPLE



Two prints, one line: "dist=23".

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6 · See also **print and move to a newline**, **join**



WHAT IT DOES

Sets the size of the printed text — small, middle, big or super big.

Possible dropdown values

Menu shown as “small”

`small` `middle` `big` `super big`

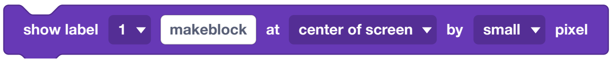
USE IT WHEN

Big for a number the class needs to read from their seats; small when several lines have to fit at once.

EXAMPLE

One large reading, visible across a room.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6 · See also **print**



WHAT IT DOES

Writes a line of text at a fixed place on the CyberPi screen, in one of four numbered slots. Writing to the same slot again replaces what was there.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

Menu shown as “center of screen”

`center of screen` `middle at top` `middle at bottom` `middle on left` `middle on right` `upper left corner` `upper right corner` `bottom left corner` `bottom right corner`

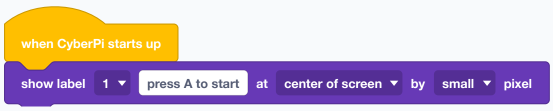
Menu shown as “small”

`small` `middle` `big` `super big`

USE IT WHEN

This is the block for a message that STAYS — an instruction, a status, a reading you want to keep an eye on. Because each slot overwrites itself, a label inside a loop updates in place instead of scrolling away.

EXAMPLE



The screen tells the driver what to do, and keeps telling them.

> Four slots, numbered 1 to 4. Two different messages in the same slot fight each other; give each its own number.

Used in G7 step 10, G7 step 11 · See also **print**, **show label 1 at x: y by small pixel**

WHAT IT DOES

The same thing, but you give an exact x and y position on the screen instead of choosing from named places.

Possible dropdown values

Menu shown as “1”

`1–8 (every whole number)`

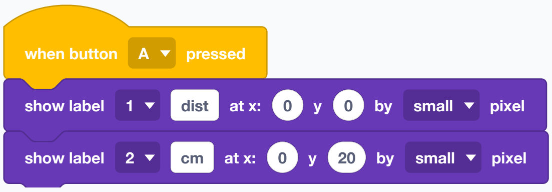
Menu shown as “small”

`small` `middle` `big` `super big`

USE IT WHEN

When the named positions do not line up the way you want — several readings side by side, or a value placed under its own heading.

EXAMPLE



Two labels, stacked exactly twenty pixels apart.

Used in G7 step 4, G7 step 5, G7 step 6, G7 step 8, G7 step 9, G7 step 10 · See also **show label 1 at center of screen by small pixel**

WHAT IT DOES

Adds one number to a line chart drawn on the screen, which scrolls as new points arrive.

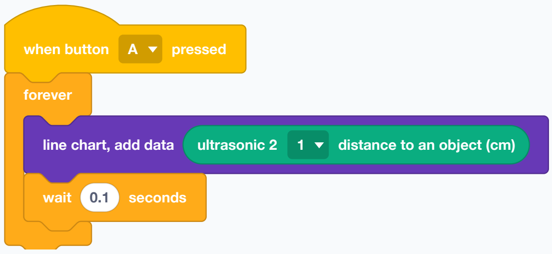
Each added number becomes the next point; older points move left as the chart fills.

Each value becomes a new point on the CyberPi screen. The points are joined; when the screen fills, older points scroll left.

USE IT WHEN

When the SHAPE of a changing reading matters more than any single value — watching a distance sensor as the robot approaches a wall shows the approach far better than a number does.

EXAMPLE



The wall arriving, drawn as a falling line.

Used in G7 step 8, G7 step 9, G7 step 10, G7 step 11 · See also **bar chart, add data**, **line chart, set spacing to pixel**

line chart, set spacing to 5 pixel

WHAT IT DOES

Sets how far apart the chart's points are drawn, in pixels.

USE IT WHEN

To fit more history on screen, or to spread a slow reading out so its changes are visible.

EXAMPLE

```
when button A ▾ pressed
line chart, set spacing to 2 pixel
```

Tighter spacing means more of the past fits on the screen.

See also **line chart, add data**

bar chart, add data 50

WHAT IT DOES

Adds one number to a bar chart on the screen.

USE IT WHEN

When you are comparing a handful of separate values rather than watching one change over time.

EXAMPLE

```
when button A ▾ pressed
bar chart, add data (quad rgb sensor 1 ▾ probe (1)R2 ▾ detects object's R value ▾)
```

A bar per reading, easy to compare at a glance.

● Used in G8 step 15 · See also **line chart, add data**

table, input excel_content at row 1 ▾ , column 1 ▾

WHAT IT DOES

Puts a value into one cell of a grid on the CyberPi screen, chosen by row and column. The table has up to four rows and three columns. “excel_content” is only placeholder text in the empty block: replace it with the text, number or sensor value you want to display. It does not read an Excel file.

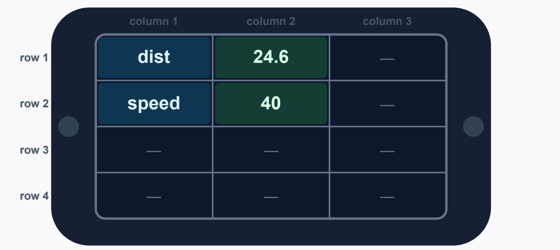
Possible dropdown values

Menu shown as “1”

1 2 3 4

Menu shown as “1”

1 2 3



Four instructions fill two labels and two live values; unused cells remain empty.

Column 1 contains the labels “dist” and “speed”; column 2 contains their current readings. Writing to the same cell again updates that cell.

USE IT WHEN

For several named readings at once — a small dashboard, where each value has its own fixed home.

EXAMPLE

```
when button A ▾ pressed
clear screen
table, input dist at row 1 ▾ , column 1 ▾
table, input speed at row 2 ▾ , column 1 ▾
forever
    table, input (ultrasonic 2 1 ▾ distance to an object (cm)) at row 1 ▾ , column 2 ▾
    table, input (encoder motor (1) EM1 ▾ 's rotational speed(RPM) ▾) at row 2 ▾ , column 2 ▾
    wait 0.1 seconds
```

Four table instructions create two rows: labels stay in column 1 while the live distance and speed readings repeatedly update column 2.

See also **show label 1 at center of screen by small pixel**

```
set brush color [ ]
```

WHAT IT DOES

Chooses the colour that later text and drawing will use, from a colour picker.

The result appears on the CyberPi's real color screen, not on the IDE stage. Source: Makeblock.

USE IT WHEN

To make one line stand out from the rest — a warning in red among readings in white.

EXAMPLE

```
when button (A ▾) pressed
set brush color [ ]
print [careful]
```

Pick a colour, then everything printed afterwards uses it.

> It changes what comes NEXT. Text already on the screen keeps the colour it was drawn in.

See also **set brush color to R G B**

```
set brush color to R (255) G (255) B (255)
```

WHAT IT DOES

The same, but mixing the colour from red, green and blue numbers instead of picking it.

USE IT WHEN

When the colour has to be worked out rather than chosen — a display that goes redder as a reading rises.

EXAMPLE

```
when button (A ▾) pressed
set brush color to R (255) G (0) B (0)
print [too close]
```

A red warning, mixed by numbers so a program can vary it.

See also **set brush color**

screen towards upside-down(-90°) ▾

WHAT IT DOES

Turns the screen image by a quarter, a half or three-quarters of a turn.

Possible dropdown values

Menu shown as “upside-down(-90°)”

`upside-down(-90°)` `left(0°)` `default(90°)` `right(180°)`

USE IT WHEN

When the CyberPi is mounted sideways or upside down on your build, so the writing comes out the right way up for whoever is reading it.

EXAMPLE

```
when CyberPi starts up
screen towards upside-down(-90 °) ▾
show label 1 ▾ ready at center of screen ▾ by small ▾ pixel
```

The board is mounted upside down; the words are not.

See also **clear screen**

clear screen

WHAT IT DOES

Wipes the whole screen — printed lines, labels, charts and all.

USE IT WHEN

At the start of a program, so you are not reading last run's leftovers, and between screens of a display that changes.

EXAMPLE

```
when button A ▾ pressed
clear screen
show label 1 ▾ ready at center of screen ▾ by small ▾ pixel
```

A clean screen before the new message.

● Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G8 step 13, G8 step 14 · See also **print and move to a newline**

LED

The five lights in a ring on the front of the CyberPi. The fastest way for a robot to say what it is thinking to a whole room.

WHAT IT DOES

Plays a ready-made light show — rainbow, spindrift, meteor, firefly — and waits until it has finished.

Possible dropdown values

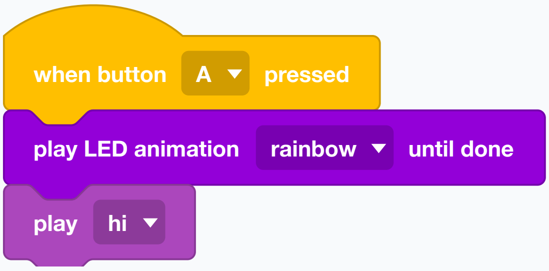
Menu shown as "rainbow"

`rainbow` `spindrift` `meteor(blue)` `meteor (green)` `helium flash(red)` `helium flash(orange)` `firefly`

USE IT WHEN

For celebration and for a "working on it" signal. One block does what a dozen LED blocks and waits would.

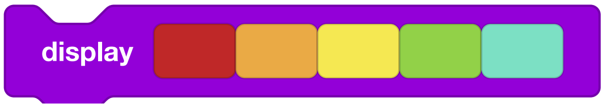
EXAMPLE



A finish-line flourish: the lights run, and only then does it chirp.

> The script waits. Nothing else in it happens while the animation plays.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6 · See also **display**



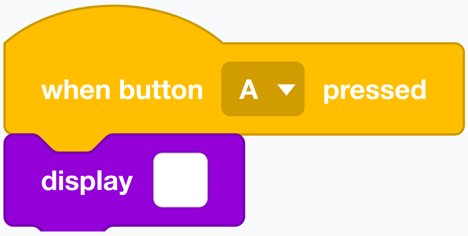
WHAT IT DOES

Sets all five LEDs at once from a little row of colour swatches you fill in on the block itself.

USE IT WHEN

When the five lights are a picture rather than one status — a level meter, a direction indicator, a pattern.

EXAMPLE



Paint the five lights individually, straight on the block.

See also **roll LEDs rightwards**

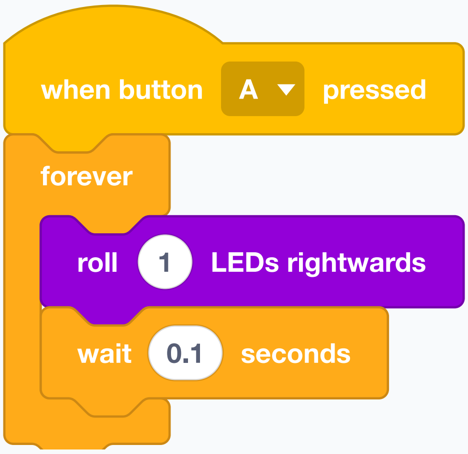
WHAT IT DOES

Shifts whatever the lights are showing round the ring by a number of places.

USE IT WHEN

To animate a pattern you have set — a dot chasing round the ring, made by lighting one LED and rolling it.

EXAMPLE



Whatever is on the ring travels round it.

See also **display**

WHAT IT DOES

A picked colour, on for a set time, then off by itself.

Possible dropdown values

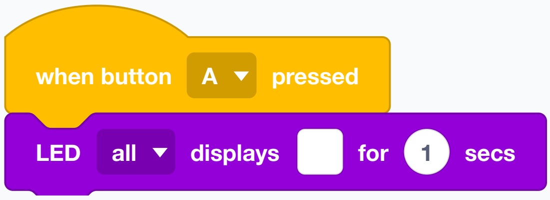
Menu shown as “all”

`all` `1` `2` `3` `4` `5`

USE IT WHEN

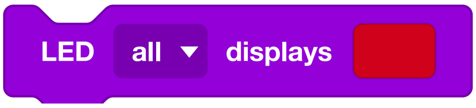
The quick version of a timed flash.

EXAMPLE



A one-second flash in the colour you picked.

See also **LED all displays R G B for secs**



WHAT IT DOES

Lights the LEDs in a colour you pick from a swatch instead of mixing from numbers.

Possible dropdown values

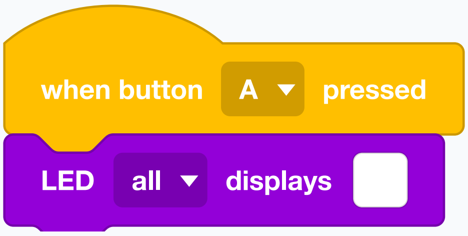
Menu shown as "all"

`all` `1` `2` `3` `4` `5`

USE IT WHEN

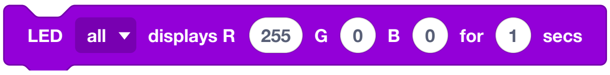
When you just want a colour and do not need a program to work it out. Quicker to reach for than the RGB version.

EXAMPLE



Click the swatch, choose a colour, done.

See also **LED all displays R G B**



WHAT IT DOES

The same, but the light comes on for a set number of seconds and then goes off by itself. The script waits while it does.

Possible dropdown values

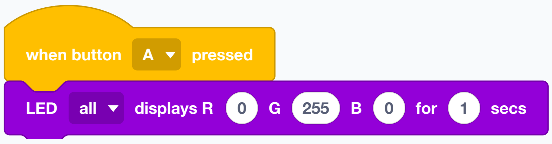
Menu shown as "all"

`all` `1` `2` `3` `4` `5`

USE IT WHEN

For a flash that does not need cleaning up afterwards — a blink to confirm a button press.

EXAMPLE



A one-second green flash, then dark again on its own.

See also **LED all displays R G B**

LED all displays R 255 G 0 B 0

WHAT IT DOES

Lights one of the five LEDs, or all of them, in a colour you mix from red, green and blue values between 0 and 255. The light stays on until something changes it.

Possible dropdown values

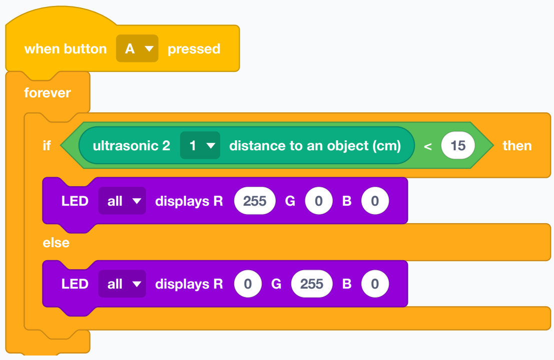
Menu shown as “all”

`all` `1` `2` `3` `4` `5`

USE IT WHEN

The workhorse status light. Because the colour is made of numbers, a program can compute it — green when clear, red when blocked, and anything between.

EXAMPLE



The robot says what it can see, without anybody reading the screen.

> 0, 0, 0 is off. 255, 255, 255 is white and very bright.

● Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G7 step 11, G7 step 18 · See also **turn off LED all**, **LED all displays R G B for secs**

increase LED brightness by 10 %

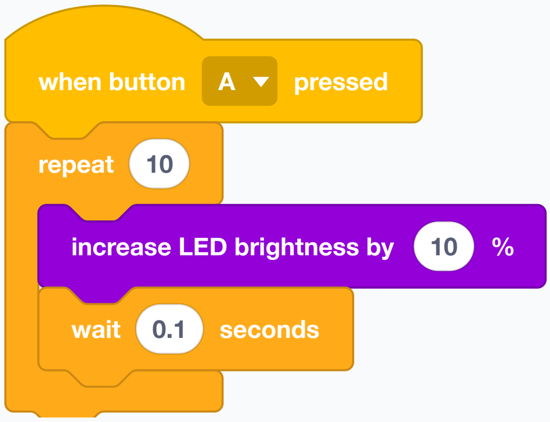
WHAT IT DOES

Raises or lowers the LED brightness by a step. A negative number dims.

USE IT WHEN

For fades, and for brightness that responds to something without your having to track the level.

EXAMPLE



The ring fades up over a second.

See also **set brightness to %**

set brightness to 30 %

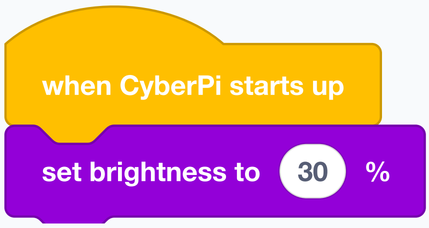
WHAT IT DOES

Sets how bright the LEDs are, as a percentage, without changing their colours.

USE IT WHEN

Once at the start, usually. Full brightness is dazzling indoors and drains the battery faster.

EXAMPLE



Comfortable to look at, and kinder to the battery.

See also **increase LED brightness by %**, **LED brightness(%)**

LED brightness(%)

WHAT IT DOES

Reports the current LED brightness as a percentage.

USE IT WHEN

When a fade needs to know where it has reached, or to show the setting on the screen.

EXAMPLE

```
when button [A ▾] pressed
print (LED brightness(%))
```

Read the brightness back.

See also **set brightness to %**

turn off LED [all ▾]

WHAT IT DOES

Switches off one LED, or all five.

Possible dropdown values

Menu shown as “all”

`all` `1` `2` `3` `4` `5`

USE IT WHEN

At the end of anything that turned them on. A robot left glowing after its program finished looks like a robot that is still running.

EXAMPLE

```
when button [B ▾] pressed
turn off LED [all ▾]
```

Lights out.

● Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6, G7 step 11, G7 step 18 · See also **LED all displays R G B**

Audio

The speaker and the microphone: built-in sounds, tones, notes, recordings, and the volume they all come out at.

See also **play hi**

WHAT IT DOES

Plays a built-in sound and waits until it has finished before the script moves on.

Possible dropdown values

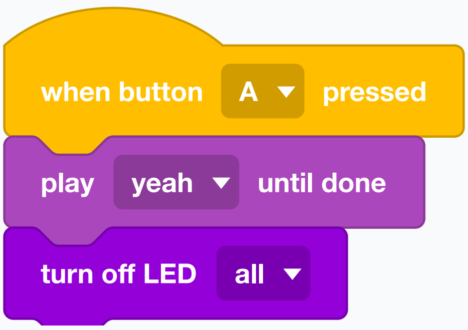
Menu shown as “hi”

`hi` `bye` `yeah` `wow` `laugh` `hum` `sad` `sigh` `annoyed` `angry` `surprised` `yummy` `curious` `embarrased` `ready` `sprint` `sleepy` `meow` `start` `switch` `beeps` `buzzing` `jump` `level-up` `low-energy` `prompt` `right` `wrong` `ring` `score` `wake` `warning` `metal-clash` `glass-clink` `inflator` `running water` `clockwork` `click` `current` `wood-hit` `iron` `drop` `bubble` `wave` `magic` `spitfire` `heartbeat`

USE IT WHEN

When the sound is the point — a countdown, a fanfare at the finish — and the next thing should not tread on it.

EXAMPLE



Let the sound finish, then go dark.

play hi ▾

WHAT IT DOES

Plays one of the CyberPi's built-in sounds and lets the script carry straight on while it plays.

Possible dropdown values

Menu shown as “hi”

`hi` `bye` `yeah` `wow` `laugh` `hum` `sad` `sigh` `annoyed` `angry` `surprised` `yummy` `curious` `embarrased` `ready` `sprint` `sleepy` `meow` `start` `switch` `beeps` `buzzing` `jump` `level-up` `low-energy` `prompt` `right` `wrong` `ring` `score` `wake` `warning` `metal-clash` `glass-clink` `inflator` `running water` `clockwork` `click` `current` `wood-hit` `iron` `drop` `bubble` `wave` `magic` `spitfire` `heartbeat`

USE IT WHEN

For a quick acknowledgement that must not hold anything up — a chirp when a button is pressed, while the robot keeps driving.

EXAMPLE

```
when button A ▾ pressed
play hi ▾
moves forward ▾ 20 cm ▾ until done
```

The chirp and the drive start together.

See also **play hi until done**

start recording

WHAT IT DOES

Starts recording sound through the CyberPi's microphone.

USE IT WHEN

When the robot should capture something to play back later — a message, a sound from the room.

EXAMPLE

```
when button A ▾ pressed
start recording
wait 3 seconds
stop recording
```

A three-second recording, started and stopped by the program.

See also **stop recording**, **play recording**

stop recording

WHAT IT DOES

Stops the recording that is in progress.

USE IT WHEN

Always, after starting one. Recording does not stop by itself.

EXAMPLE

```
when button B ▾ pressed
stop recording
```

B ends the recording.

See also **start recording**

play recording until done

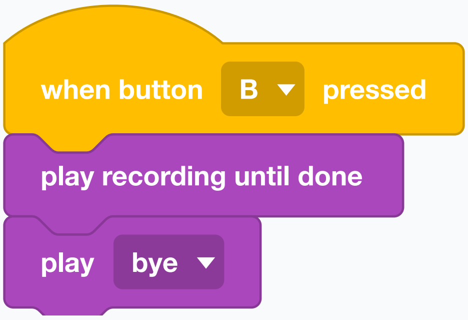
WHAT IT DOES

Plays back the last recording and waits until it has finished.

USE IT WHEN

When nothing should happen until the message has been heard.

EXAMPLE



The message plays out fully before the sign-off.

See also **play recording**

play recording

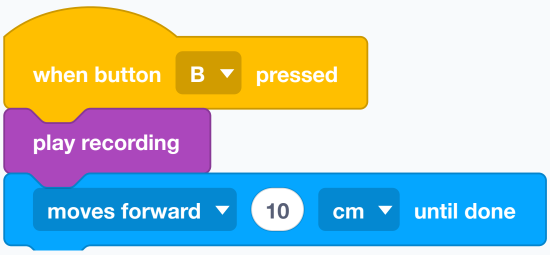
WHAT IT DOES

Plays back the last recording and carries on immediately.

USE IT WHEN

When the playback should happen alongside something else.

EXAMPLE



The robot talks and drives at the same time.

Used in G7 step 3, G7 step 4, G7 step 5, G7 step 6 · See also **play recording until done**

play note 60 for 0.25 beat

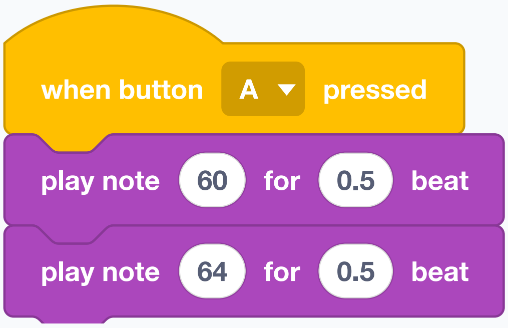
WHAT IT DOES

Plays a musical note for a number of beats, with the pitch chosen from a keyboard on the block.

USE IT WHEN

For tunes rather than beeps. Notes and beats are easier to compose with than frequencies and seconds.

EXAMPLE



Two notes of a tune.

See also **play snare for beat**

```
play [snare ▾] for (0.25) beat
```

WHAT IT DOES

Plays a drum sound for a number of beats.

Possible dropdown values

Menu shown as “snare”

`snare` `bass drum` `side stick` `crash cymbal` `open hi-hat` `closed hi-hat` `tambourine` `hand clap` `claves`

USE IT WHEN

For rhythm — marking time, counting laps audibly.

EXAMPLE

```
when button [A ▾] pressed
repeat (4)
    play [snare ▾] for (0.25) beat
end
```

Four beats on the snare.

See also **play note for beat**

```
increase audio speed by (10) %
```

WHAT IT DOES

Nudges the playback speed up or down by a step.

USE IT WHEN

For an effect that changes gradually.

EXAMPLE

```
when button [A ▾] pressed
repeat (5)
    increase audio speed by (20) %
    play [hi ▾] until done
end
```

The same sound, faster each time.

See also **set audio speed to %**

```
set audio speed to (100) %
```

WHAT IT DOES

Sets the playback speed of sounds as a percentage; below 100 is slower and deeper, above is faster and higher.

USE IT WHEN

For character — a slowed voice, a chipmunk squeak.

EXAMPLE

```
when button [A ▾] pressed
set audio speed to (50) %
play [hi ▾] until done
```

The same sound, half speed.

See also **audio speed**

```
(audio speed)
```

WHAT IT DOES

Reports the current playback speed as a percentage.

USE IT WHEN

To display or restore the setting.

EXAMPLE

```
when button [A ▾] pressed
print (audio speed)
```

Read the speed back.

See also **set audio speed to %**

```
increase volume by (10) %
```

WHAT IT DOES

Raises or lowers the volume by a step; a negative number turns it down.

USE IT WHEN

For a volume the user can adjust with a button.

EXAMPLE

```
when joystick [pulled↑ ▾]
increase volume by (10) %
```

Push the stick up to turn it up.

See also **set volume to %**

```
set volume to (30) %
```

WHAT IT DOES

Sets the speaker volume as a percentage.

USE IT WHEN

Once at the start. Twenty robots at full volume in one classroom is a genuine problem.

EXAMPLE

```
when CyberPi starts up
set volume to (40) %
```

Loud enough for one group, quiet enough for the room.

See also **increase volume by %**, **volume(%)**

volume(%)

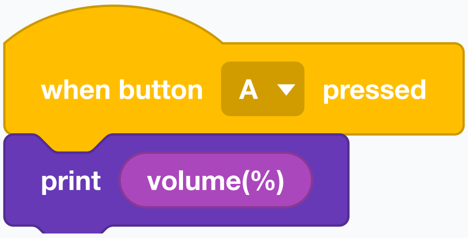
WHAT IT DOES

Reports the current volume as a percentage.

USE IT WHEN

To show the setting on screen while somebody adjusts it.

EXAMPLE



Show what the volume is now.

See also **set volume to %**

play sound at 700 Hz

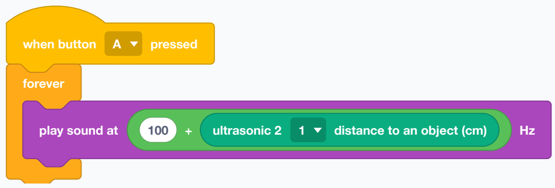
WHAT IT DOES

Sounds a tone at a frequency you give in hertz, and carries on immediately. Higher numbers are higher notes.

USE IT WHEN

When the pitch should mean something — a tone that rises as the robot gets closer to a wall is a sensor you can hear.

EXAMPLE



The robot hums lower as it nears a wall.

See also **play sound at Hz for secs**

play sound at 700 Hz for 1 secs

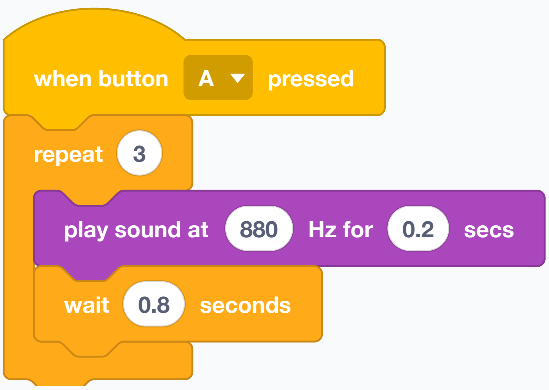
WHAT IT DOES

Sounds a tone for a set number of seconds, and waits while it does.

USE IT WHEN

For a beep of a definite length — a metronome, a countdown tick.

EXAMPLE



Three short pips, one a second.

See also **play sound at Hz**

stop all sounds

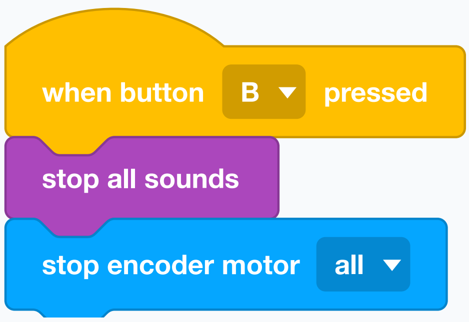
WHAT IT DOES

Stops everything that is making a sound, at once.

USE IT WHEN

To cut off a long sound early, and as part of a stop button.

EXAMPLE



Silence and stillness together.

See also **play hi**

Sensing

The CyberPi's own inputs — its buttons, its joystick, its microphone and light sensor, its timer, and facts about the board itself.

joystick pulled↑ ?

WHAT IT DOES

Reports true while the CyberPi's own joystick is being held in a direction.

Possible dropdown values

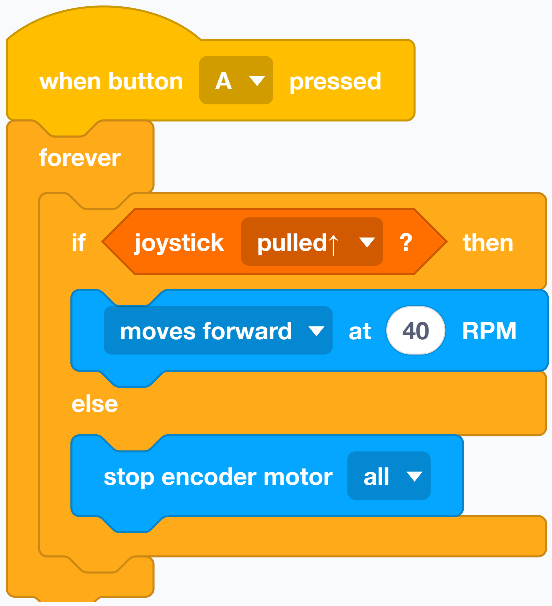
Menu shown as "pulled↑"

`pulled↑` `pulled↓` `pulled←` `pulled→` `middle pressed` `any direction`

USE IT WHEN

For steering or menu navigation using the onboard stick, inside a loop.

EXAMPLE



The robot drives only while the stick is held forward.

See also **when joystick pulled↑**

joystick pulled↑ counts

WHAT IT DOES

Reports how many times the onboard joystick has been pushed in a direction since the count was reset.

Possible dropdown values

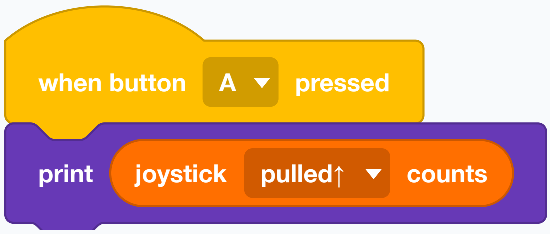
Menu shown as "pulled↑"

`pulled↑` `pulled↓` `pulled←` `pulled→` `middle pressed`

USE IT WHEN

For stepping through a list of choices with the stick.

EXAMPLE



How many times has the stick been pushed up?

See also **reset joystick pulled↑ counts**

reset joystick pulled↑ counts

WHAT IT DOES

Sets a joystick direction's count back to zero.

Possible dropdown values

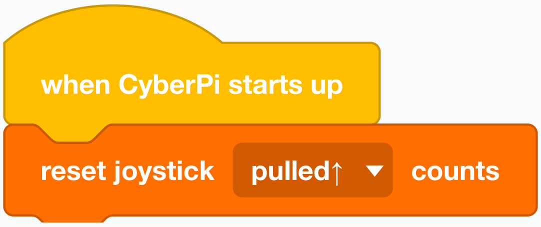
Menu shown as “pulled↑”

`pulled↑` `pulled↓` `pulled←` `pulled→` `middle pressed` `any direction`

USE IT WHEN

Before starting a fresh count.

EXAMPLE



Zero the counter at boot.

See also **joystick pulled↑ counts**

button A pressed?

WHAT IT DOES

Reports true while button A or B is being held down. It is a question, not a hat — it tells you the state right now.

Possible dropdown values

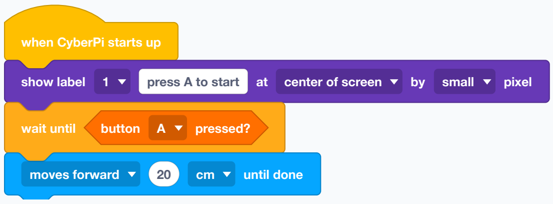
Menu shown as “A”

`A` `B` `any button`

USE IT WHEN

Inside a loop or a wait, when you want to ask rather than be interrupted. It is what "wait until button A pressed" is built from.

EXAMPLE



The gate that keeps an uploaded program still until somebody is ready.

See also **when button A pressed**, **wait until**

WHAT IT DOES

Reports how many times a button has been pressed since the count was last reset.

Possible dropdown values

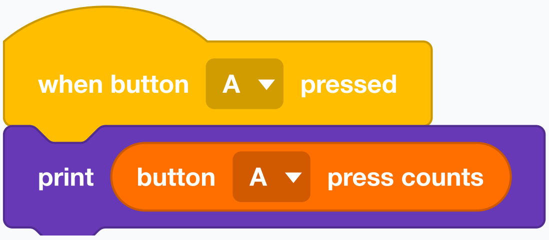
Menu shown as “A”

`A` `B`

USE IT WHEN

To cycle through modes without a variable of your own — press A three times for mode three.

EXAMPLE



Show the running total of presses.

See also **reset button A press counts**

WHAT IT DOES

Sets a button's press count back to zero.

Possible dropdown values

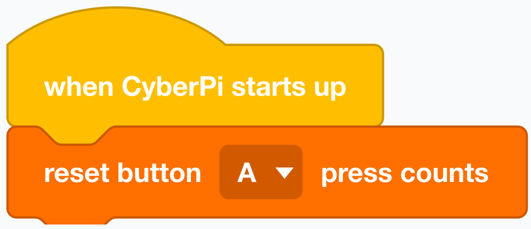
Menu shown as “A”

`A` `B` `any button`

USE IT WHEN

At the start of a program, and whenever a mode cycle should begin again.

EXAMPLE



Start counting from nothing.

See also **button A press counts**

loudness

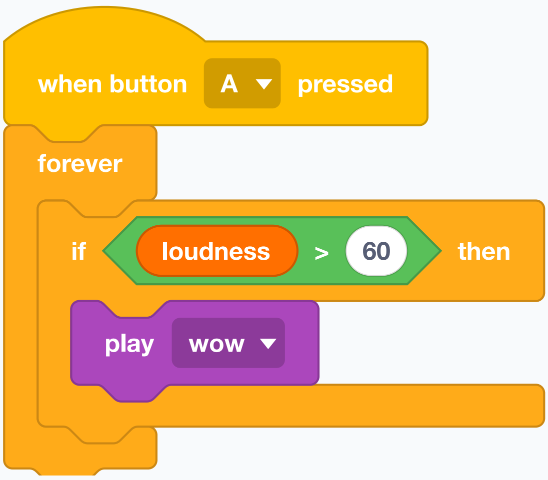
WHAT IT DOES

Reports how loud the room is right now, as a number. Quiet is near zero; a clap is high.

USE IT WHEN

For sound-triggered behaviour — start on a clap, stop when the room goes quiet. Also the honest way to pick a threshold: print it in a real classroom before choosing a number.

EXAMPLE



The robot answers a loud noise.

A quiet room and a noisy hall need different numbers. Measure before you choose.

Used in G7 step 8, G7 step 9, G7 step 10, G7 step 11 · See also **when light value >**

ambient light intensity

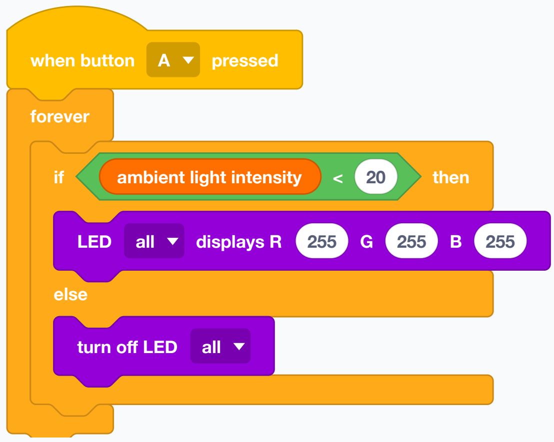
WHAT IT DOES

Reports how bright the light falling on the CyberPi is, as a number.

USE IT WHEN

For light-following and light-avoiding, and for noticing a change — a hand passing over the robot drops the reading sharply.

EXAMPLE



The robot lights its own way when the room goes dark.

Room lighting changes through the day. A threshold that worked this morning may not work this afternoon.

Used in G7 step 8, G7 step 9, G7 step 10, G7 step 11 · See also **loudness**

timer(s)

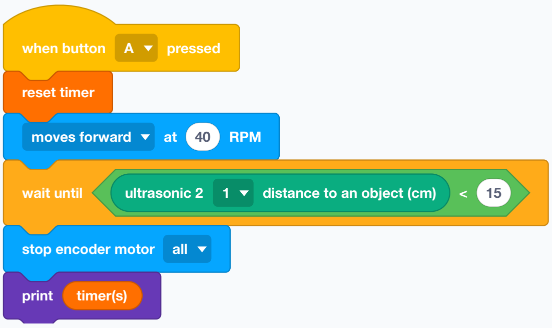
WHAT IT DOES

Reports how many seconds have passed since the timer was last reset.

USE IT WHEN

For anything measured in elapsed time — a lap time, a timeout, a behaviour that changes after a while.

EXAMPLE



How long did it take to reach the wall?

Used in G8 step 14, G8 step 15 · See also **reset timer**

reset timer

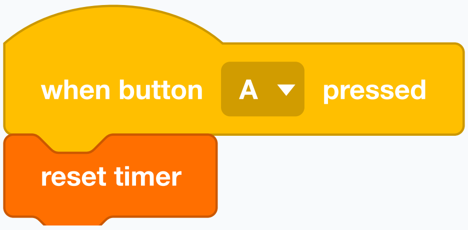
WHAT IT DOES

Sets the timer back to zero.

USE IT WHEN

Immediately before whatever you are timing.

EXAMPLE



Start the clock.

Used in G8 step 14, G8 step 15 · See also **timer(s)**

hostname

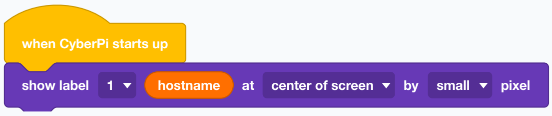
WHAT IT DOES

Reports the CyberPi's own name, the one it shows when connecting.

USE IT WHEN

In a classroom of identical robots, to put each one's name on its own screen.

EXAMPLE



Every robot says which one it is.

See also **battery level(%)**

battery level(%)

WHAT IT DOES

Reports one fact about the board itself, chosen from its dropdown — battery level, MAC address, Bluetooth name, system language.

Possible dropdown values

Menu shown as “battery level(%)”

`battery level(%)` `MAC address` `Bluetooth name` `system language` `extension name` `external battery (%)`

USE IT WHEN

Battery level is the useful one. A robot behaving strangely is very often a robot that is nearly flat.

> The whole block is one dropdown, so which fact it reports is chosen on the block itself rather than typed into a slot.

See also **hostname**

NOT USED IN THIS COURSE

WHAT IT DOES

Reports whether the robot is in the automatic or manual stage of a MakeX competition round.

Possible dropdown values

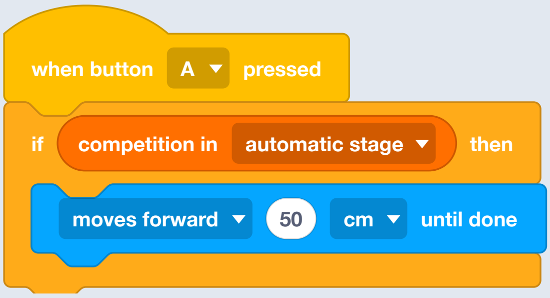
Menu shown as “automatic stage”

`automatic stage` `manual stage`

USE IT WHEN

Only in MakeX competitions, which run programs in two stages.

EXAMPLE



Behave differently depending on which stage the round is in.

Motion Sensing

The motion sensor inside the board: which way it is tilted, how hard it is shaken, and how far it has turned.

LIVE MODE ONLY

control ( ) to follow CyberPi with sensitivity [low(0.2) ▾]

WHAT IT DOES

Links a sprite on the mBlock stage to the CyberPi's movement, so tilting the board moves the sprite.

Possible dropdown values

Menu shown as “low(0.2)”

`low(0.2)` `middle(0.4)` `high(0.6)`

USE IT WHEN

Live mode only, when the robot is being used as a controller for something on the screen.

EXAMPLE

```
when green flag clicked
control ( ) to follow CyberPi with sensitivity [middle(0.4) ▾]
```

The board becomes a tilt controller for the stage.

See also **tilted forward ?**

<tilted forward ▾ ?>

WHAT IT DOES

Reports true while the CyberPi is being held at a particular angle.

Possible dropdown values

Menu shown as “tilted forward”

`tilted forward` `tilted backward` `tilted left` `tilted right` `screen up` `screen down`

USE IT WHEN

Inside a loop, to check an orientation rather than be interrupted by it. Also the way to notice the robot has tipped over.

EXAMPLE

```
when button [A ▾] pressed
forever
    if <tilted forward ▾ ?> then
        [moves forward ▾] at (40) RPM
    else
        stop encoder motor [all ▾]
```

Tilt the robot forward and it drives; level it and it stops.

See also **when CyberPi tilted left**, **tilted forward angle (°)**

wave up ▾ detected?

WHAT IT DOES

Reports true when a gesture has just been made — a wave in a direction, or a twist.

Possible dropdown values

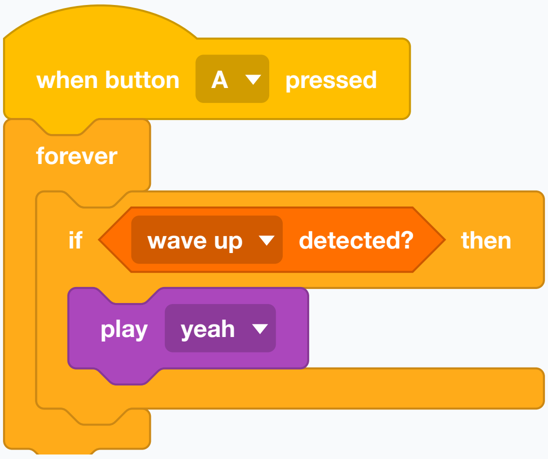
Menu shown as “wave up”

`wave up` `wave down` `wave left` `wave right` `rotate clockwise` `rotate counterclockwise` `freefall` `shaken`

USE IT WHEN

Inside a loop, when a gesture should be one of several things you are checking.

EXAMPLE



A flick upwards gets an answer.

See also **when wave left detected**

shaking strength

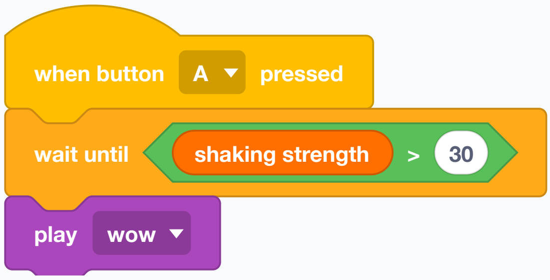
WHAT IT DOES

Reports how hard the board is being shaken, as a number.

USE IT WHEN

For a shake-to-start or shake-to-reset, where a threshold is friendlier than a yes-or-no gesture.

EXAMPLE



Give it a good shake to set it off.

See also **wave up detected?**

waving direction (°)

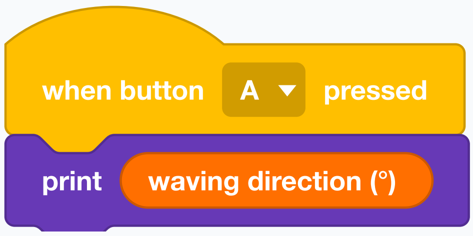
WHAT IT DOES

Reports the direction of a wave, in degrees.

USE IT WHEN

When a gesture should point somewhere rather than pick from four names.

EXAMPLE



Which way was that wave?

See also **waving speed**

waving speed

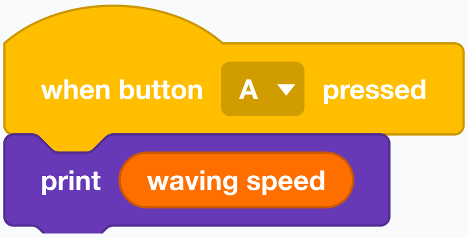
WHAT IT DOES

Reports how fast the board is being waved.

USE IT WHEN

To tell a gentle movement from a sharp one.

EXAMPLE



How energetic was that?

See also **waving direction (°)**

tilted forward angle (°)

WHAT IT DOES

Reports the actual angle of tilt in degrees, rather than a yes or no.

Possible dropdown values

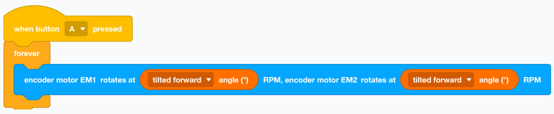
Menu shown as “tilted forward”

`tilted forward`
`tilted backward(pitch)` `tilted left`
`tilted right(roll)`
`rotate clockwise`
`rotate counterclockwise(yaw)`

USE IT WHEN

When how much matters — a robot that drives proportionally to how far it is tipped, or a build that must stay level.

EXAMPLE



Tip it further and it drives faster.

See also **tilted forward ?**

motion sensor [x ▾] acceleration(m/s^2)

WHAT IT DOES

Reports acceleration along one axis in metres per second squared. At rest, the downward axis reads about 9.8 because gravity never stops pulling.

Possible dropdown values

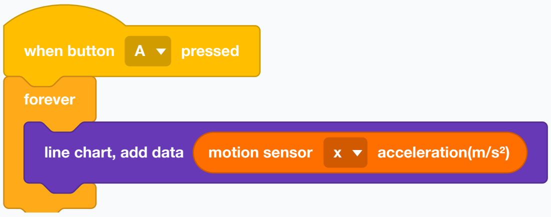
Menu shown as “x”

x | y | z

USE IT WHEN

For detecting knocks, drops and sudden stops — a crash is a large brief spike.

EXAMPLE



Watch the bumps as the robot drives over them.

See also **angle speed around x axis(°/s)**

angle speed around [x ▾] axis(°/s)

WHAT IT DOES

Reports how fast the board is turning about one axis, in degrees per second.

Possible dropdown values

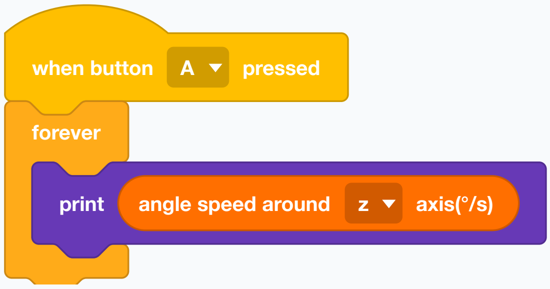
Menu shown as “x”

x | y | z

USE IT WHEN

To measure a spin as it happens, rather than the total after it.

EXAMPLE



How fast is the robot spinning right now?

See also **rotated angle around x axis (°)**

rotated angle around x axis (°)

WHAT IT DOES

Reports the total angle turned about one axis since the last reset, adding up as the robot turns.

Possible dropdown values

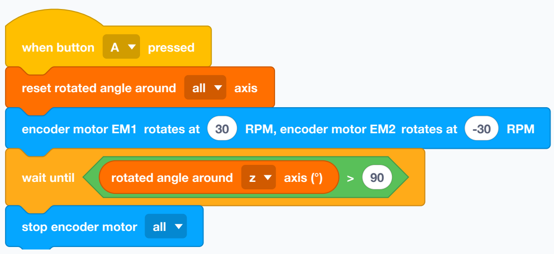
Menu shown as “x”

`x` `y` `z`

USE IT WHEN

For turning by a known angle without trusting the wheels. On a slippery floor the wheels lie about the turn and this does not.

EXAMPLE



Spin until the robot has really turned a right angle, then stop.

> The total keeps growing until reset, so reset before you measure.

See also **reset rotated angle around all axis**

reset rotated angle around all axis

WHAT IT DOES

Sets the accumulated rotation back to zero for one axis, or all three.

Possible dropdown values

Menu shown as “all”

`all` `x` `y` `z`

USE IT WHEN

Immediately before a turn you intend to measure.

EXAMPLE

Zero the angle counters.

See also **rotated angle around x axis (°)**

reset yaw angle

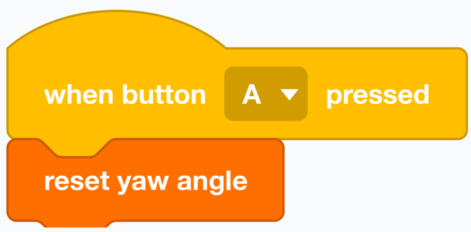
WHAT IT DOES

Sets the robot's heading to zero, making whichever way it is pointing the new forward.

USE IT WHEN

At the start of a run, so headings are measured from where the robot actually began rather than from where it was switched on.

EXAMPLE



This direction is now north.

See also **rotated angle around x axis (°)**

LAN

Robot-to-robot messages over the local network. For robots working together in the same room.

broadcast message `message` on LAN

WHAT IT DOES

Sends a named message to every CyberPi on the same local network channel.

USE IT WHEN

When robots must work together — one starts, and the rest follow.

EXAMPLE

```
when button A ▾ pressed
broadcast go on LAN
```

One robot tells all the others to begin.

> Both robots must be set to the same LAN channel, or neither hears the other.

See also **when receiving broadcast on LAN**, **set LAN channel to 1**

broadcast message `message` with value `1` on LAN

WHAT IT DOES

Sends a named message and a number or word along with it.

USE IT WHEN

When the message needs content — not just "go" but "go at speed 40".

EXAMPLE

```
when button A ▾ pressed
broadcast speed with value 40 on LAN
```

The instruction and its number travel together.

● Used in G8 step 14, G8 step 15 · See also **LAN broadcast value received**

when receiving `message` broadcast on LAN

WHAT IT DOES

A hat block that runs when a named message arrives over the network.

USE IT WHEN

On the receiving robot, to act on what another robot said.

EXAMPLE

```
when receiving go broadcast on LAN
moves forward ▾ 20 cm ▾ until done
```

This robot moves when another one says go.

● Used in G8 step 14, G8 step 15 · See also **broadcast message on LAN**

LAN broadcast (message) value received

WHAT IT DOES

Reports the value that came with the last message of that name.

USE IT WHEN

Straight after the matching hat, to read what was sent.

EXAMPLE

```
when receiving [speed] broadcast on LAN
moves forward ▾ at (LAN broadcast [speed] value received) RPM
```

The sender chose the speed; this robot uses it.

● Used in G8 step 14, G8 step 15 · See also **broadcast message with value on LAN**

set LAN channel to (1 ▾)

WHAT IT DOES

Chooses which network channel the robot talks and listens on.

Possible dropdown values

Menu shown as "1"

`1` `6 (default)` `11`

USE IT WHEN

When two groups in one room must not hear each other. Put each team on its own channel and their messages stop crossing.

EXAMPLE

```
when CyberPi starts up
set LAN channel to (11 ▾)
```

This team keeps to channel 11.

See also **broadcast message on LAN**

Upload Mode Broadcast

Messages between an uploaded program and mBlock on the computer — the way to see what a robot running on its own is doing.

send upload mode message (message)

WHAT IT DOES

Sends a named message from an uploaded program back to mBlock on the computer.

USE IT WHEN

To watch what an uploaded robot is doing. Once a program is uploaded it is on its own, and this is the thread back to the screen.

EXAMPLE

```
when button (A ▾) pressed
send upload mode message (started)
```

The robot tells the computer it has begun.

See also **send upload mode message with value**

send upload mode message (message) with value (1)

WHAT IT DOES

Sends a named message with a value attached.

USE IT WHEN

For sending readings out of an uploaded program — logging a sensor while the robot runs untethered.

EXAMPLE

```
when button (A ▾) pressed
forever
    send upload mode message (dist) with value (ultrasonic 2 (1 ▾) distance to an object (cm))
    wait (0.5) seconds
```

A live feed of the distance sensor, twice a second.

See also **upload mode message value**

when receiving upload mode message (message)

WHAT IT DOES

A hat block that runs when a named upload-mode message arrives.

USE IT WHEN

On the receiving side of the link.

EXAMPLE

```
when receiving upload mode message (go)
moves forward ▾ (20) (cm ▾) until done
```

Act on a message from the other side.

See also **send upload mode message**

upload mode message (message) value

WHAT IT DOES

Reports the value carried by the last message of that name.

USE IT WHEN

Straight after the matching hat, to read what came with it.

EXAMPLE

```
when receiving upload mode message (speed)
moves forward ▾ at (upload mode message (speed) value) RPM
```

The message carried the speed to use.

See also **send upload mode message with value**

AI

Cloud services: speaking, listening, translating, and the Wi-Fi they all depend on. Every one of these needs a network, a signed-in mBlock account, and Upload mode.

connect to Wi-Fi ssid password password

WHAT IT DOES

Joins a Wi-Fi network, given its name and password. The script waits while it connects.

USE IT WHEN

First, before anything that needs the internet. Every AI and IoT block — speaking, listening, translating, the weather — needs a network first.

EXAMPLE

```
when CyberPi starts up
connect to Wi-Fi ClassroomWiFi password letmein
wait until network connected?
play hi ▾
```

Connect, wait until it really is connected, then say so.

Upload mode only. A school network that needs a browser login page will not work — the robot has no browser.

Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 ·
See also **network connected?**

network connected?

WHAT IT DOES

Reports true once the robot is on a network.

USE IT WHEN

Straight after connecting, to wait properly instead of guessing a number of seconds.

EXAMPLE

```
when CyberPi starts up
connect to Wi-Fi ClassroomWiFi password letmein
wait until network connected?
show label 1 ▾ online at center of screen ▾ by small ▾ pixel
```

Wait for the real thing rather than a hopeful delay.

See also **connect to Wi-Fi password**

connect to Wi-Fi again

WHAT IT DOES

Joins the last network again without your having to give the name and password a second time.

USE IT WHEN

After a drop-out, or at the start of a program that has connected before.

EXAMPLE

```
when button B ▾ pressed
connect to Wi-Fi again
```

A reconnect button for when the network drops.

See also **connect to Wi-Fi password**

disconnect from Wi-Fi

WHAT IT DOES

Leaves the network.

USE IT WHEN

To save battery, or to prove that something works without the internet.

EXAMPLE

when button (B ▾) pressed
disconnect from Wi-Fi

Off the network.

See also **connect to Wi-Fi again**

speak (auto ▾) (hello world)

WHAT IT DOES

Reads text aloud through the speaker, using a voice generated in the cloud.

Possible dropdown values

Menu shown as “auto”

`auto`

USE IT WHEN

When the robot should say something a screen cannot — a result across a noisy room, a greeting, an answer to a question it was asked.

EXAMPLE

when button (A ▾) pressed
connect to Wi-Fi (ClassroomWiFi) password (letmein)
wait until <network connected?>
speak (auto ▾) (hello everyone)

Connect first, then talk.

> Needs Wi-Fi, an mBlock account signed in, AND Upload mode. If the block looks greyed out or hatched in the editor, one of those three is missing.

● Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 · See also **recognize (1) Chinese secs**

recognize (1) Chinese 3 secs

WHAT IT DOES

Listens through the microphone for a number of seconds and sends what it hears to the cloud to be turned into text. The script waits while it listens.

Possible dropdown values

Menu shown as “(1) Chinese”

`(1) Chinese` `(2) Chinese(Taiwan)` `(3) Cantonese` `(4) English` `(5) French` `(6) German` `(7) Spanish` `(8) Italian` `(9) Japanese` `(10) Portuguese` `(11) Russian` `(12) Korean`

USE IT WHEN

To take a spoken command. Pair it with the result block, which holds what came back.

EXAMPLE

```
when button A pressed
recognize (1) Chinese 3 secs
print speech recognition result
```

Listen for three seconds, then show what was heard.

> Choose the language on the block. Same three requirements as speaking: Wi-Fi, signed in, Upload mode.

Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 ·
See also **speech recognition result**

speech recognition result

WHAT IT DOES

Reports the text that came back from the last listen.

USE IT WHEN

Straight after recognising, to test what was said.

EXAMPLE

```
when button A pressed
recognize (1) Chinese 3 secs
if < speech recognition result contains go ?> then
    moves forward 20 cm until done
```

Say the word and the robot moves. "contains" is kinder than "equals" here, because people say more than the one word.

> It holds the LAST result. Read it before listening again.

Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 ·
See also **recognize (1) Chinese secs**, **contains ?**

translate hello into Chinese

```
translate [hello] into [Chinese ▾]
```

WHAT IT DOES

Sends text to the cloud and reports it back in another language.

Possible dropdown values

Menu shown as “Chinese”

`Chinese` `English` `Traditional Chinese` `Japanese` `French` `German` `Spanish` `Portuguese` `Russian` `Korean` `Italian` `Dutch`

USE IT WHEN

For a bilingual robot — hear a command in one language, answer in another.

EXAMPLE

```
when button [A ▾] pressed
speak [auto ▾] (translate [hello] into [English ▾])
```

Translate, then say the translation out loud.

Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 · See also **speak auto**

IoT

The internet: messages that can cross a building, and data from the outside world such as weather, air quality and the real time.

connect to Wi-Fi ssid password password

WHAT IT DOES

Joins a Wi-Fi network, given its name and password. The script waits while it connects.

USE IT WHEN

First, before anything that needs the internet. Every AI and IoT block — speaking, listening, translating, the weather — needs a network first.

EXAMPLE

```
when CyberPi starts up
connect to Wi-Fi ClassroomWiFi password letmein
wait until network connected?
play hi ▾
```

Connect, wait until it really is connected, then say so.

> Upload mode only. A school network that needs a browser login page will not work — the robot has no browser.

● Used in G8 step 17, G8 step 18, G8 step 19, G8 step 20 · See also **network connected?**

network connected?

WHAT IT DOES

Reports true once the robot is on a network.

USE IT WHEN

Straight after connecting, to wait properly instead of guessing a number of seconds.

EXAMPLE

```
when CyberPi starts up
connect to Wi-Fi ClassroomWiFi password letmein
wait until network connected?
show label 1 ▾ online at center of screen ▾ by small ▾ pixel
```

Wait for the real thing rather than a hopeful delay.

See also **connect to Wi-Fi password**

connect to Wi-Fi again

WHAT IT DOES

Joins the last network again without your having to give the name and password a second time.

USE IT WHEN

After a drop-out, or at the start of a program that has connected before.

EXAMPLE

```
when button B ▾ pressed
connect to Wi-Fi again
```

A reconnect button for when the network drops.

See also **connect to Wi-Fi password**

disconnect from Wi-Fi

WHAT IT DOES

Leaves the network.

USE IT WHEN

To save battery, or to prove that something works without the internet.

EXAMPLE

```
when button B ▾ pressed
disconnect from Wi-Fi
```

Off the network.

See also **connect to Wi-Fi again**

send user cloud broadcast message

WHAT IT DOES

Sends a named message over the internet to any robot signed in to the same account.

USE IT WHEN

When the robots are not in the same room. LAN broadcast is for robots side by side; this one goes anywhere with a network.

EXAMPLE

```
when button A ▾ pressed
send user cloud broadcast go
```

A message that can cross a building.

> Wi-Fi, a signed-in account, and Upload mode.

See also **when receiving user cloud broadcast**, **broadcast message on LAN**

send user cloud broadcast message with value 1

WHAT IT DOES

Sends a named cloud message with a value attached.

USE IT WHEN

To send a reading, not just a signal.

EXAMPLE

```
when button A ▾ pressed
send user cloud broadcast temp with value 21
```

The message carries its number with it.

See also **user cloud broadcast value received**

when receiving user cloud broadcast message

WHAT IT DOES

A hat block that runs when a named cloud message arrives.

USE IT WHEN

On the receiving robot.

EXAMPLE

```
when receiving user cloud broadcast go
play hi ▾
```

The robot answers a message from somewhere else entirely.

See also **send user cloud broadcast**

user cloud broadcast [message] value received

WHAT IT DOES

Reports the value that came with the last cloud message of that name.

USE IT WHEN

Straight after the matching hat.

EXAMPLE

```
when receiving user cloud broadcast [temp]
print (user cloud broadcast [temp] value received)
```

Show the number that arrived.

See also **send user cloud broadcast with value**

[ ] highest temperature (°C) ▾

WHAT IT DOES

Reports today's temperature for a place you name, fetched from the internet.

Possible dropdown values

Menu shown as "highest temperature (°C)"

`highest temperature (°C)` `lowest temperature(°C)` `highest temperature(°F)` `lowest temperature(°F)` `weather` `humidity`

USE IT WHEN

For a weather station project, or any program whose behaviour should depend on the world outside the classroom.

> The place goes in the block's first slot, before the dropdown, so there is no plain-text way to write this one out — build it in the editor. Needs Wi-Fi and Upload mode.

See also **air quality AQI**, **connect to Wi-Fi password**

air quality [ ] AQI ▾

WHAT IT DOES

Reports an air-quality figure — the AQI index, or the PM2.5 and PM10 particle counts — for a named place.

Possible dropdown values

Menu shown as "AQI"

`AQI` `PM2.5` `PM10` `CO` `SO₂` `NO₂`

USE IT WHEN

For environmental projects, and for comparing the reading outside with one taken in the room.

EXAMPLE

```
when button [A ▾] pressed
print (air quality [Phnom Penh] [AQI ▾])
```

Today's air quality, on the robot's screen.

> Wi-Fi and Upload mode.

See also **highest temperature (°C)**

WHAT IT DOES

Reports the sunrise or sunset time for a named place.

Possible dropdown values

Menu shown as “sunrise”

`sunrise` `sunset`

Menu shown as “time”

`time` `hour` `minute`

USE IT WHEN

For a robot that should behave differently after dark.

> Like the weather block, the place comes first in the block, so this one has to be assembled in the editor rather than written out. Needs Wi-Fi and Upload mode.

See also **highest temperature (°C)**

WHAT IT DOES

Reports one part of the current time for a chosen time zone, in one block.

Possible dropdown values

Menu shown as “(0) UTC”

`(0) UTC` `(1) UTC+1` `(2) UTC+2` `(3) UTC+3` `(4) UTC+4` `(5) UTC+5` `(6) UTC+6` `(7) UTC+7` `(8) UTC+8` `(9) UTC+9` `(10) UTC+10` `(11) UTC+11` `(12) UTC+12` `(-12) UTC-12` `(-11) UTC-11` `(-10) UTC-10` `(-9) UTC-9` `(-8) UTC-8` `(-7) UTC-7` `(-6) UTC-6` `(-5) UTC-5` `(-4) UTC-4` `(-3) UTC-3` `(-2) UTC-2` `(-1) UTC-1`

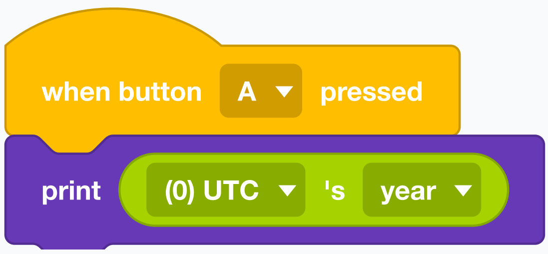
Menu shown as “year”

`year` `month` `day` `week` `hour` `minute` `second`

USE IT WHEN

When you want a single value and do not need the whole timestamp.

EXAMPLE



One number, straight from the network clock.

See also **year of date and time**

(0) UTC 's date and time

WHAT IT DOES

Fetches the current date and time for a time zone from the internet.

Possible dropdown values

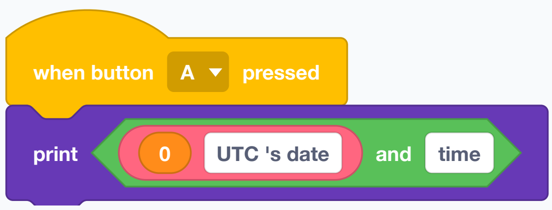
Menu shown as “(0) UTC”

`(0) UTC` `(1) UTC+1` `(2) UTC+2` `(3) UTC+3` `(4) UTC+4` `(5) UTC+5` `(6) UTC+6` `(7) UTC+7` `(8) UTC+8` `(9) UTC+9` `(10) UTC+10` `(11) UTC+11` `(12) UTC+12` `(-12) UTC-12` `(-11) UTC-11` `(-10) UTC-10` `(-9) UTC-9` `(-8) UTC-8` `(-7) UTC-7` `(-6) UTC-6` `(-5) UTC-5` `(-4) UTC-4` `(-3) UTC-3` `(-2) UTC-2` `(-1) UTC-1`

USE IT WHEN

Before reading any of the separate parts. The robot has no clock of its own that survives being switched off.

EXAMPLE



The real date and time, from the network.

See also **year of date and time**

year of date and time

WHAT IT DOES

Reports one part of the time that was last fetched — the year, the month, the date, the hour.

Possible dropdown values

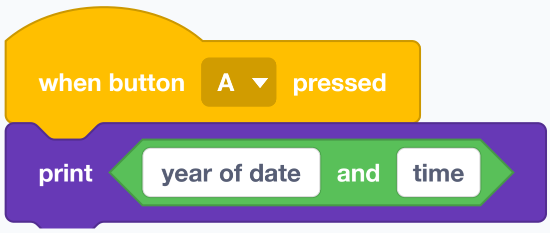
Menu shown as “year”

`year` `month` `date` `week` `hour` `minute` `second`

USE IT WHEN

After fetching, to use one piece rather than the whole string.

EXAMPLE



Just the year, out of the whole timestamp.

See also **(0) UTC 's date and time**

Magic emoji

Faces for the CyberPi screen — generated by AI, drawn by you, or picked from a set of ready-made animations.

Set AI Emojis to

```
Set AI Emojis to [ ]
```

WHAT IT DOES

Shows a picture on the CyberPi screen that was generated by the AI Emoji Workshop — the button at the top of this drawer, which asks the cloud to draw faces from a description.

USE IT WHEN

To give the robot an expression without drawing one yourself. The picture goes in the slot on the block; the workshop is where you make it.

EXAMPLE

```
when button [A ▾] pressed
Set AI Emojis to [ ]
```

Whichever generated face you dropped into the slot appears on the screen.

> The workshop needs Wi-Fi and a signed-in account. The block itself only shows a picture that has already been made.

See also **Set Animojis to Cute**

Set Animojis to Cute

```
Set Animojis to [Cute ▾]
```

WHAT IT DOES

Plays one of eight ready-made animated faces — cute, unhappy, sleepy, surprised, cool, angry, happy, in love.

Possible dropdown values

Menu shown as “Cute”

`Cute` `Unhappy` `Sleepy` `Surprise` `Cool` `Angry` `Happy` `Love`

USE IT WHEN

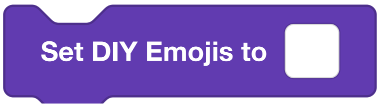
The quickest way to give the robot a mood. No drawing and no network needed.

EXAMPLE

```
when button [A ▾] pressed
Set Animojis to [Happy ▾]
```

The robot grins.

See also **Set DIY Emojis to**



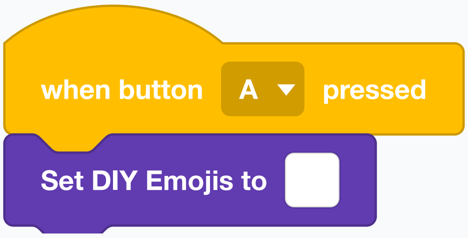
WHAT IT DOES

Shows a face you drew yourself, pixel by pixel, on the little grid built into the block.

USE IT WHEN

When you want a specific expression and no internet. Drawing sixteen by sixteen pixels is also a good lesson in how little a screen needs to say something.

EXAMPLE



Your own drawing, straight on the robot's face.

See also **Set AI Emojis to**

Sprites

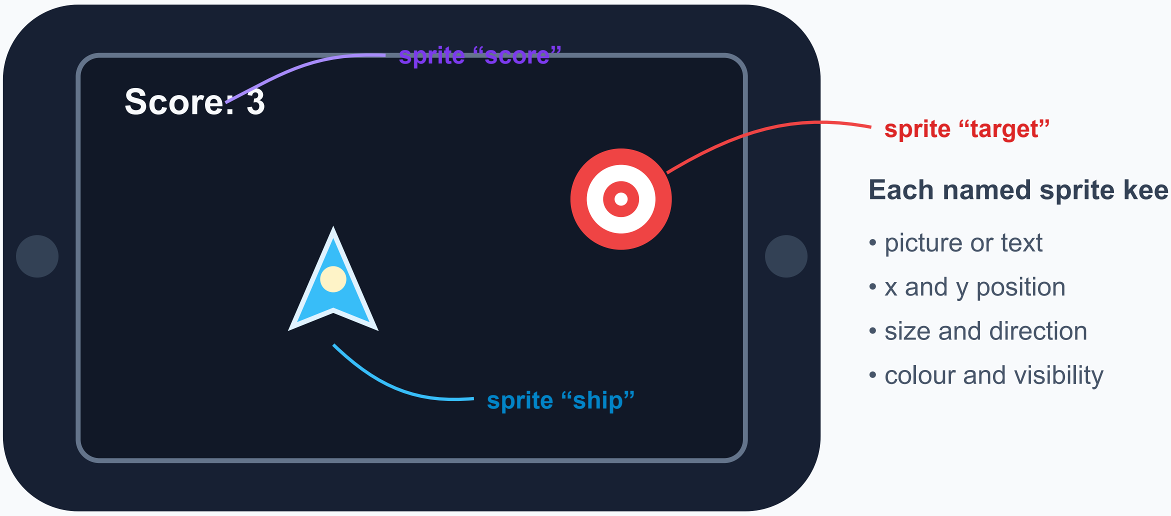
> Named pictures on the screen that you move about independently, rather than lines of text that scroll. The basis of games and dials on the robot itself.

Mini-course: what is a sprite?

A **sprite** is a named visual object that a program can control independently. It can be text, a built-in icon, a picture drawn on a pixel grid, or a QR code. Unlike ordinary printed text, a sprite remembers its own position and appearance.

Which screen?

The blocks in this **Sprites** drawer create sprites on the **CyberPi's physical 128×128 screen**. They do not create or move the Panda on the mBlock IDE stage. The separate "follow mBlock sprite" sensing block is an exception that explicitly refers to an IDE-stage sprite.



Moving "ship" does not move "score" or "target".

Why use sprites?

Suppose a game screen contains a score, a moving spaceship and a target. With printed text, redrawing one item can disturb the others. With sprites, the program can move **ship**, recolour **target**, and replace the text in **score** independently. Sprites are useful for games, gauges, arrows, status icons and small animated interfaces.

The basic workflow

Step	What the program does
1. Create and name	Use a "set sprite … show to …" block. The name—such as `ship`—is how later blocks find that same object.
2. Position	Place it at an x and y coordinate, or move it by a number of pixels.
3. Change	Rotate it, resize it, recolour it, flip it or change its layer.
4. Interact	Ask whether it touches another sprite or the edge—useful for games.
5. Hide or delete	Hide keeps the sprite ready to show again; delete removes it.

Names connect the blocks

```
create sprite “ship” → move sprite “ship” → rotate sprite “ship” → hide sprite “ship”
```

Every block using the name **ship** acts on the same sprite. A block using the name **target** acts on a different one. Choose short, meaningful names so it is obvious which screen object each block controls.

```
set backdrop to [colour]
```

WHAT IT DOES

Fills the whole screen behind the sprites with a colour you pick.

USE IT WHEN

To set a mood or make a warning unmissable — a whole red screen says more than a red word.

EXAMPLE

```
when button (A ▾) pressed
set backdrop to [colour]
```

The whole screen changes colour.

See also **set backdrop to R: G: B:**

```
set backdrop to R: (0) G: (0) B: (0)
```

WHAT IT DOES

Fills the screen behind the sprites with a colour mixed from numbers.

USE IT WHEN

When the background colour has to be computed — a screen that reddens as something gets worse.

EXAMPLE

```
when button (A ▾) pressed
set backdrop to R: (255) G: (0) B: (0)
```

A screen a program can colour by itself.

See also **set backdrop to**

```
set sprite [show ▾] to [ ]
```

WHAT IT DOES

Makes a sprite out of a picture you draw yourself on a small grid on the block.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

For a face, an arrow, an icon of your own — anything the built-in pictures do not cover.

EXAMPLE

```
when button [A ▾] pressed
set sprite [face show ▾] to [ ]
```

Draw it on the block; it appears on the screen.

See also **set sprite show to music**

```
set sprite [show ▾] to [music ▾]
```

WHAT IT DOES

Makes a sprite from one of the CyberPi's built-in icons.

Possible dropdown values

Menu shown as “show”

`show` `hide`

Menu shown as “music”

`music` `picture` `video` `clock` `play` `pause` `next` `previous` `sound` `temperature` `light` `motion` `house` `gear` `list` `tick symbol` `cross sign` `power off` `refresh` `trash bin` `download` `sunny` `cloudy` `rain` `snow` `train` `rocket` `truck` `car` `water drop` `distance` `flame` `magnet` `gas` `vision` `color` `overcast` `sandstorm` `fog`

USE IT WHEN

When a ready-made symbol will do and you would rather not draw one.

EXAMPLE

```
when button [A ▾] pressed
set sprite [icon show ▾] to [music]
```

A built-in symbol, no drawing needed.

See also **set sprite show to**

```
set sprite [show ▾] to [hello world]
```

WHAT IT DOES

Makes a named sprite out of a piece of text and puts it on the screen. From then on, other sprite blocks can move, colour and hide it by that name.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

When a word has to move about rather than sit in a fixed slot — a score that slides in, a label that follows something.

EXAMPLE

```
when button [A ▾] pressed
set sprite [score show ▾] to (0)
sprite [score show ▾] goes to x (40) y (20)
```

A named piece of text, put exactly where you want it.

See also **sprite show goes to x y**, **show label 1 at center of screen by small pixel**

```
set sprite [show ▾] to QR code [www.makeblock.com]
```

WHAT IT DOES

Makes a sprite that is a QR code containing whatever text you give it.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

To hand something to a phone — a link, a robot's name, a result. It turns the robot's screen into something a visitor can scan.

EXAMPLE

```
when button [A ▾] pressed
set sprite [code show ▾] to QR code [hello]
```

A scannable code on the robot's face.

See also **set sprite show to**

sprite [show ▾] **flips** [left-right ▾]

WHAT IT DOES

Flips a sprite left-to-right or top-to-bottom.

Possible dropdown values

Menu shown as "show"

`show` `hide`

Menu shown as "left-right"

`left-right` `up-down`

USE IT WHEN

To reuse one drawing facing both ways — one arrow, pointing either direction.

EXAMPLE

```
when button [A ▾] pressed
sprite [arrow show ▾] flips [left-right ▾]
```

The same arrow, now pointing the other way.

See also **sprite show rotates ° clockwise**

delete sprite [show ▾]

WHAT IT DOES

Removes a sprite for good.

Possible dropdown values

Menu shown as "show"

`show` `hide`

USE IT WHEN

When it will not be needed again. Hiding is better if it might.

EXAMPLE

```
when button [B ▾] pressed
delete sprite [dot show ▾]
```

Gone for good.

See also **show sprite**

set anchor point of sprite show ▾ **to** top left ▾

WHAT IT DOES

Chooses which part of a sprite counts as its position — its middle, a corner, an edge.

Possible dropdown values

Menu shown as “show”

`show` `hide`

Menu shown as “top left”

`top left` `top middle` `top right` `middle left` `center` `middle right` `bottom left` `bottom center` `bottom right`

USE IT WHEN

When things must line up. Text anchored at its left edge starts in the same place however long it gets; text anchored at its middle shifts as it grows.

EXAMPLE

```
when button A ▾ pressed
set anchor point of sprite score show ▾ to top left ▾
```

Anchored at the corner, so it grows rightwards instead of spreading both ways.

See also **sprite show goes to x y**

sprite show ▾ **moves** right (x) ▾ 8 **pixels**

WHAT IT DOES

Moves a sprite a number of pixels from where it is now, in a direction you choose.

Possible dropdown values

Menu shown as “show”

`show` `hide`

Menu shown as “right (x)”

`right (x)` `down (y)` `left` `right (x)`

USE IT WHEN

For animation by steps, where each move follows the last.

EXAMPLE

```
when button A ▾ pressed
repeat 10
    sprite dot show ▾ moves right (x) ▾ 5 pixels
    wait 0.1 seconds
```

The sprite walks across the screen.

See also **sprite show goes to x y**

```
sprite [show ▾] goes to x (64) y (64)
```

WHAT IT DOES

Puts a sprite at an exact x and y position on the screen.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

For laying a screen out deliberately, and for showing a value as a position — a marker that moves as a reading changes.

EXAMPLE

```
when button [A ▾] pressed
forever
    sprite [dot show ▾] goes to x (ultrasonic 2 [1 ▾] distance to an object (cm)) y (30)
```

A dot that slides across the screen as the wall gets nearer.

See also **sprite show moves right (x) pixels**

```
sprite [show ▾] goes to random position
```

WHAT IT DOES

Jumps a sprite to a random place on the screen.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

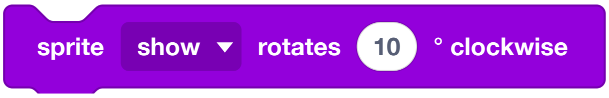
For games — a target that appears somewhere new each round.

EXAMPLE

```
when button [A ▾] pressed
sprite [target show ▾] goes to random position
```

A new place every press.

See also **pick random to**



WHAT IT DOES

Turns a sprite by a number of degrees from its current direction.

Possible dropdown values

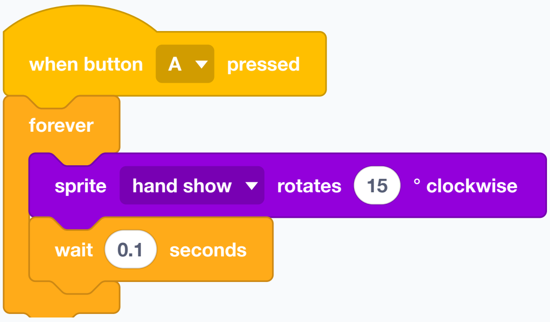
Menu shown as “show”

`show` `hide`

USE IT WHEN

For spinning something, and for a needle that turns as a reading changes.

EXAMPLE



A hand sweeping round like a clock.

See also **sprite show points in direction °**

WHAT IT DOES

Points a sprite in an exact direction rather than turning it by a step.

Possible dropdown values

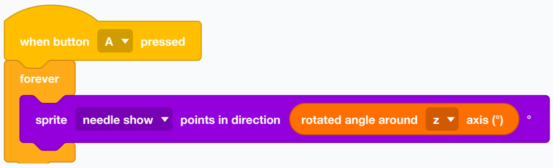
Menu shown as “show”

`show` `hide`

USE IT WHEN

For a compass or a dial, where the angle IS the value.

EXAMPLE



A needle on screen that follows how far the robot has turned.

See also **sprite show rotates ° clockwise**

set sprite show ▾ size to (100) %

WHAT IT DOES

Scales a sprite, as a percentage of its normal size.

Possible dropdown values

Menu shown as "show"

`show` `hide`

USE IT WHEN

To make something grow or shrink — a bar that fills, an alert that swells.

EXAMPLE

```
when button A ▾ pressed
set sprite alert show ▾ size to (200) %
```

Twice as big, to catch the eye.

See also **x position of sprite show**

set sprite show ▾ color to [ ]

WHAT IT DOES

Recolours a sprite using a colour you pick.

Possible dropdown values

Menu shown as "show"

`show` `hide`

USE IT WHEN

To change what a sprite means without redrawing it — the same shape, red for danger and green for clear.

EXAMPLE

```
when button A ▾ pressed
set sprite dot show ▾ color to [ ]
```

Pick a colour on the block.

See also **set sprite show color to R: G: B:**

set sprite show color to R: 255 G: 255 B: 255

WHAT IT DOES

Recolours a sprite from red, green and blue numbers.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

When the colour must be worked out — redder as a reading rises.

EXAMPLE

```
when button A pressed
set sprite dot show color to R: 255 G: 0 B: 0
```

A colour a program can compute.

See also **set sprite show color to**

reset sprite show to default color

WHAT IT DOES

Puts a sprite back to the colour it was drawn in.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

After a temporary highlight.

EXAMPLE

```
when button B pressed
reset sprite dot show to default color
```

Back to normal.

See also **set sprite show color to**

```
show ▾ sprite [ ]
```

WHAT IT DOES

Shows or hides a sprite without deleting it. A hidden sprite keeps its position and can come back.

Possible dropdown values

Menu shown as “show”

`show` `hide`

USE IT WHEN

For blinking, and for screens that change — hide this group, show that one.

EXAMPLE

```
when button A ▾ pressed
forever
    show ▾ sprite dot
    wait 0.5 seconds
    hide ▾ sprite dot
    wait 0.5 seconds
```

A blinking dot.

See also **delete sprite show**

```
sprite show ▾ goes to front ▾ layer
```

WHAT IT DOES

Sends a sprite right to the front or right to the back of the pile.

Possible dropdown values

Menu shown as “show”

`show` `hide`

Menu shown as “front”

`front` `back`

USE IT WHEN

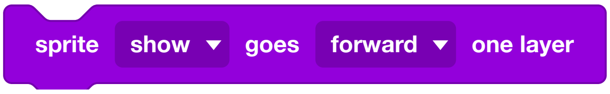
When sprites overlap and one must be readable over the others.

EXAMPLE

```
when button A ▾ pressed
sprite alert show ▾ goes to front ▾ layer
```

The warning sits on top of everything else.

See also **sprite show goes forward one layer**



WHAT IT DOES

Moves a sprite one step up or down the pile.

Possible dropdown values

Menu shown as “show”

`show` `hide`

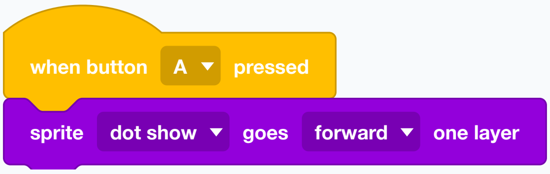
Menu shown as “forward”

`forward` `backward`

USE IT WHEN

For finer control than front-or-back when three or more sprites overlap.

EXAMPLE



One step nearer the front.

See also **sprite show goes to front layer**

WHAT IT DOES

Reports true when two sprites are overlapping.

Possible dropdown values

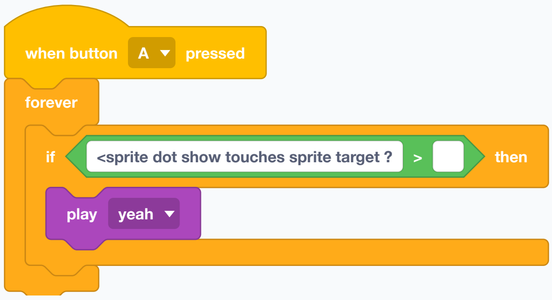
Menu shown as “show”

`show` `hide`

USE IT WHEN

For games on the robot's own screen — catching, dodging, collecting.

EXAMPLE



A hit is worth a noise.

See also **sprite show on edge?**

sprite show on edge?

WHAT IT DOES

Reports true when a sprite has reached the edge of the screen.

Possible dropdown values

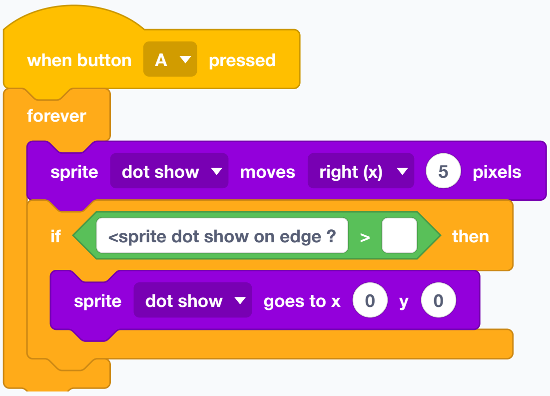
Menu shown as “show”

`show` `hide`

USE IT WHEN

To bounce something back, or to stop it disappearing.

EXAMPLE



It walks to the edge and starts again from the middle.

See also **sprite show touches sprite ?**

x position of sprite show

WHAT IT DOES

Reports one fact about a sprite — its x, its y, its direction, its size.

Possible dropdown values

Menu shown as “x position”

`x position` `y position` `direction` `size` `anchor point`

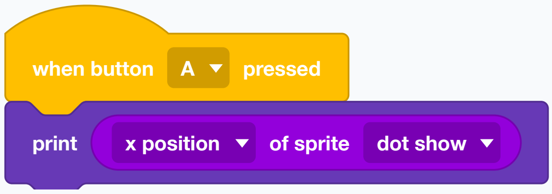
Menu shown as “show”

`show` `hide`

USE IT WHEN

When a movement must be worked out from where the sprite already is.

EXAMPLE



Where has it got to?

See also **sprite show goes to x y**

color of x: 64 y: 64 on the display is R: 0 G: 0 B: 0

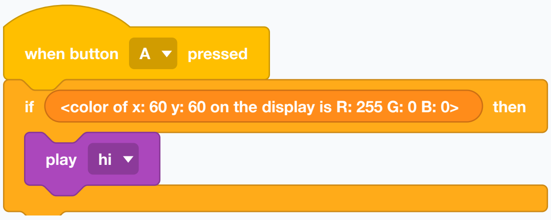
WHAT IT DOES

Reports true when the pixel at a given point on the screen is a particular colour.

USE IT WHEN

A different way to test a collision — by what is drawn rather than by which sprites overlap.

EXAMPLE



Ask the screen itself what colour it is showing.

See also **sprite show touches sprite ?**

force rendering

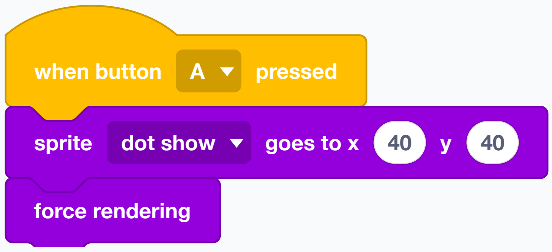
WHAT IT DOES

Redraws the screen right now, rather than waiting for the next automatic refresh.

USE IT WHEN

When several changes must appear together, or when an animation looks jerky because the screen is updating at its own pace rather than yours.

EXAMPLE



Move it and show the move immediately.

See also **sprite show goes to x y**